

Hugging Face is way more fun with friends and colleagues! 😊 [Join an organization](#)[Dismiss this message](#)

Transformers documentation

Utilities for Generation ▾You are viewing *main* version, which requires [installation from source](#). If you'd like regular pip install, checkout the latest stable version ([v4.43.2](#)).

Utilities for Generation

This page lists all the utility functions used by `generate()`.

Generate Outputs

The output of `generate()` is an instance of a subclass of `ModelOutput`. This output is a data structure containing all the information returned by `generate()`, but that can also be used as tuple or dictionary.

Here's an example:

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel

tokenizer = GPT2Tokenizer.from_pretrained("openai-community/gpt2")
model = GPT2LMHeadModel.from_pretrained("openai-community/gpt2")

inputs = tokenizer("Hello, my dog is cute and ", return_tensors="pt")
generation_output = model.generate(**inputs, return_dict_in_generate=True, output_scores=True)
```

The `generation_output` object is a `GenerateDecoderOnlyOutput`, as we can see in the documentation of that class below, it means it has the following attributes:

- `sequences`: the generated sequences of tokens
- `scores` (optional): the prediction scores of the language modelling head, for each generation step
- `hidden_states` (optional): the hidden states of the model, for each generation step
- `attentions` (optional): the attention weights of the model, for each generation step

Here we have the `scores` since we passed along `output_scores=True`, but we don't have `hidden_states` and `attentions` because we didn't pass `output_hidden_states=True` or `output_attentions=True`.

You can access each attribute as you would usually do, and if that attribute has not been returned by the model, you will get `None`. Here for instance `generation_output.scores` are all the generated prediction scores of the language modeling head, and `generation_output.attentions` is `None`.

When using our `generation_output` object as a tuple, it only keeps the attributes that don't have `None` values. Here, for instance, it has two elements, `loss` then `logits`, so

```
generation_output[:2]
```

will return the tuple (`generation_output.sequences`, `generation_output.scores`) for instance.

When using our `generation_output` object as a dictionary, it only keeps the attributes that don't have `None` values. Here, for instance, it has two keys that are `sequences` and `scores`.

We document here all output types.

PyTorch

`class transformers.generation.GenerateDecoderOnlyOutput`

```
( sequences: LongTensor = None, scores: Optional = None, logits: Optional = None, attentions: Optional = None,
hidden_states: Optional = None, past_key_values: Optional = None )
```

Expand 6 parameters

Outputs of decoder-only generation models, when using non-beam methods.

`class transformers.generation.GenerateEncoderDecoderOutput`

```
( sequences: LongTensor = None, scores: Optional = None, logits: Optional = None, encoder_attentions: Optional = None,
encoder_hidden_states: Optional = None, decoder_attentions: Optional = None, cross_attentions: Optional = None,
decoder_hidden_states: Optional = None, past_key_values: Optional = None )
```

Expand 9 parameters

Outputs of encoder-decoder generation models, when using non-beam methods.

class transformers.generation.GenerateBeamDecoderOnlyOutput



```
( sequences: LongTensor = None, sequences_scores: Optional = None, scores: Optional = None, logits: Optional = None,
beam_indices: Optional = None, attentions: Optional = None, hidden_states: Optional = None, past_key_values: Optional =
None )
```

Expand 8 parameters

Outputs of decoder-only generation models, when using beam methods.

class transformers.generation.GenerateBeamEncoderDecoderOutput



```
( sequences: LongTensor = None, sequences_scores: Optional = None, scores: Optional = None, logits: Optional = None,
beam_indices: Optional = None, encoder_attentions: Optional = None, encoder_hidden_states: Optional = None,
decoder_attentions: Optional = None, cross_attentions: Optional = None, decoder_hidden_states: Optional = None,
past_key_values: Optional = None )
```

Expand 11 parameters

Outputs of encoder-decoder generation models, when using beam methods.

TensorFlow

class transformers.generation.TFGreedySearchEncoderDecoderOutput



```
( sequences: Tensor = None, scores: Optional = None, encoder_attentions: Optional = None, encoder_hidden_states: Optional = None, decoder_attentions: Optional = None, cross_attentions: Optional = None, decoder_hidden_states: Optional = None )
```

Expand 7 parameters

Base class for outputs of encoder-decoder generation models using greedy search. Hidden states and attention weights of the decoder (respectively the encoder) can be accessed via the `encoder_attentions` and the `encoder_hidden_states` attributes (respectively the `decoder_attentions` and the `decoder_hidden_states` attributes)

`class transformers.generation.TFGreedySearchDecoderOnlyOutput`

```
( sequences: Tensor = None, scores: Optional = None, attentions: Optional = None, hidden_states: Optional = None )
```

Parameters

- **sequences** (`tf.Tensor` of shape `(batch_size, sequence_length)`) — The generated sequences. The second dimension (`sequence_length`) is either equal to `max_length` or shorter if all batches finished early due to the `eos_token_id`.
- **scores** (`tuple(tf.Tensor)` *optional*, returned when `output_scores=True` is passed or when `config.output_scores=True`) — Processed prediction scores of the language modeling head (scores for each vocabulary token before SoftMax) at each generation step. Tuple of `tf.Tensor` with up to `max_new_tokens` elements (one element for each generated token), with each tensor of shape `(batch_size, config.vocab_size)`.
- **attentions** (`tuple(tuple(tf.Tensor))`, *optional*, returned when `output_attentions=True` is passed or `config.output_attentions=True`) — Tuple (one element for each generated token) of tuples (one element for each layer of the decoder) of `tf.Tensor` of shape `(batch_size, num_heads, generated_length, sequence_length)`.
- **hidden_states** (`tuple(tuple(tf.Tensor))`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple (one element for each generated token) of tuples (one element for each layer of the decoder) of `tf.Tensor` of shape `(batch_size, generated_length, hidden_size)`.

Base class for outputs of decoder-only generation models using greedy search.

`class transformers.generation.TFSampleEncoderDecoderOutput`

```
( sequences: Tensor = None, scores: Optional = None, encoder_attentions: Optional = None, encoder_hidden_states: Optional = None, decoder_attentions: Optional = None, cross_attentions: Optional = None, decoder_hidden_states: Optional = None )
```

Expand 7 parameters

Base class for outputs of encoder-decoder generation models using sampling. Hidden states and attention weights of the decoder (respectively the encoder) can be accessed via the `encoder_attentions` and the `encoder_hidden_states` attributes (respectively the `decoder_attentions` and the `decoder_hidden_states` attributes)

 `class transformers.generation.TFSampleDecoderOnlyOutput` 

```
( sequences: Tensor = None, scores: Optional = None, attentions: Optional = None, hidden_states: Optional = None )
```

Expand 4 parameters

Base class for outputs of decoder-only generation models using sampling.

 `class transformers.generation.TFBeamSearchEncoderDecoderOutput` 

```
( sequences: Tensor = None, sequences_scores: Optional = None, scores: Optional = None, beam_indices: Optional = None,
encoder_attentions: Optional = None, encoder_hidden_states: Optional = None, decoder_attentions: Optional = None,
cross_attentions: Optional = None, decoder_hidden_states: Optional = None )
```

[Expand 9 parameters](#)

Base class for outputs of encoder-decoder generation models using beam search. Hidden states and attention weights of the decoder (respectively the encoder) can be accessed via the `encoder_attentions` and the `encoder_hidden_states` attributes (respectively the `decoder_attentions` and the `decoder_hidden_states` attributes)

 `class transformers.generation.TFBeamSearchDecoderOnlyOutput` 

```
( sequences: Tensor = None, sequences_scores: Optional = None, scores: Optional = None, beam_indices: Optional = None,
  attentions: Optional = None, hidden_states: Optional = None )
```

[Expand 6 parameters](#)

Base class for outputs of decoder-only generation models using beam search.

 `class transformers.generation.TFBeamSampleEncoderDecoderOutput` 

```
( sequences: Tensor = None, sequences_scores: Optional = None, scores: Optional = None, beam_indices: Optional = None,
  encoder_attentions: Optional = None, encoder_hidden_states: Optional = None, decoder_attentions: Optional = None,
  cross_attentions: Optional = None, decoder_hidden_states: Optional = None )
```

[Expand 9 parameters](#)

Base class for outputs of encoder-decoder generation models using beam sampling. Hidden states and attention weights of the decoder (respectively the encoder) can be accessed via the `encoder_attentions` and the `encoder_hidden_states` attributes (respectively the `decoder_attentions` and the `decoder_hidden_states` attributes)

class `transformers.generation.TFBeamSampleDecoderOnlyOutput`

```
( sequences: Tensor = None, sequences_scores: Optional = None, scores: Optional = None, beam_indices: Optional = None,
  attentions: Optional = None, hidden_states: Optional = None )
```

Expand 6 parameters

Base class for outputs of decoder-only generation models using beam sample.

class `transformers.generation.TFContrastiveSearchEncoderDecoderOutput`

```
( sequences: Tensor = None, scores: Optional = None, encoder_attentions: Optional = None, encoder_hidden_states: Optional
  = None, decoder_attentions: Optional = None, cross_attentions: Optional = None, decoder_hidden_states: Optional = None )
```

Expand 7 parameters

Base class for outputs of encoder-decoder generation models using contrastive search. Hidden states and attention weights of the decoder (respectively the encoder) can be accessed via the `encoder_attentions` and the `encoder_hidden_states` attributes (respectively the `decoder_attentions` and the `decoder_hidden_states` attributes)

 `class transformers.generation.TFContrastiveSearchDecoderOnlyOutput`

```
( sequences: Tensor = None, scores: Optional = None, attentions: Optional = None, hidden_states: Optional = None )
```

Parameters

- **sequences** (`tf.Tensor` of shape `(batch_size, sequence_length)`) — The generated sequences. The second dimension (`sequence_length`) is either equal to `max_length` or shorter if all batches finished early due to the `eos_token_id`.
- **scores** (`tuple(tf.Tensor)` *optional*, returned when `output_scores=True` is passed or when `config.output_scores=True`) — Processed prediction scores of the language modeling head (scores for each vocabulary token before SoftMax) at each generation step. Tuple of `tf.Tensor` with up to `max_new_tokens` elements (one element for each generated token), with each tensor of shape `(batch_size, config.vocab_size)`.
- **attentions** (`tuple(tuple(tf.Tensor))`, *optional*, returned when `output_attentions=True` is passed or `config.output_attentions=True`) — Tuple (one element for each generated token) of tuples (one element for each layer of the decoder) of `tf.Tensor` of shape `(batch_size, num_heads, generated_length, sequence_length)`.
- **hidden_states** (`tuple(tuple(tf.Tensor))`, *optional*, returned when `output_hidden_states=True` is passed or when `config.output_hidden_states=True`) — Tuple (one element for each generated token) of tuples (one element for each layer of the decoder) of `tf.Tensor` of shape `(batch_size, generated_length, hidden_size)`.

Base class for outputs of decoder-only generation models using contrastive search.

FLAX `class transformers.generation.FlaxSampleOutput`

```
( sequences: Array = None )
```

Parameters

- **sequences** (`jnp.ndarray` of shape `(batch_size, max_length)`) — The generated sequences.

Flax Base class for outputs of decoder-only generation models using sampling.

 `replace`

```
( **updates )
```

“Returns a new object replacing the specified fields with new values.

 `class transformers.generation.FlaxGreedySearchOutput`

```
( sequences: Array = None )
```

Parameters

- **sequences** (`jnp.ndarray` of shape `(batch_size, max_length)`) — The generated sequences.

Flax Base class for outputs of decoder-only generation models using greedy search.

 `replace`

(****updates**)

“Returns a new object replacing the specified fields with new values.

class `transformers.generation.FlaxBeamSearchOutput`

(`sequences: Array = None, scores: Array = None`)

Parameters

- **sequences** (`jnp.ndarray` of shape `(batch_size, max_length)`) — The generated sequences.
- **scores** (`jnp.ndarray` of shape `(batch_size,)`) — The scores (log probabilities) of the generated sequences.

Flax Base class for outputs of decoder-only generation models using greedy search.

`replace`

(****updates**)

“Returns a new object replacing the specified fields with new values.

LogitsProcessor

A [LogitsProcessor](#) can be used to modify the prediction scores of a language model head for generation.

PyTorch

class `transformers.AlternatingCodebooksLogitsProcessor`

(`input_start_len: int, semantic_vocab_size: int, codebook_size: int`)

Parameters

- **input_start_len** (`int`) — The length of the initial input sequence.
- **semantic_vocab_size** (`int`) — Vocabulary size of the semantic part, i.e number of tokens associated to the semantic vocabulary.
- **codebook_size** (`int`) — Number of tokens associated to the codebook.

[LogitsProcessor](#) enforcing alternated generation between the two codebooks of Bark.

This logits processor is exclusively compatible with [Bark](#)'s fine submodel. See the model documentation for examples.

`__call__`

(`input_ids: LongTensor, scores: FloatTensor`)

class `transformers.ClassifierFreeGuidanceLogitsProcessor`

(guidance_scale)

Parameters

- **guidance_scale** (float) — The guidance scale for classifier free guidance (CFG). CFG is enabled by setting `guidance_scale > 1`. Higher guidance scale encourages the model to generate samples that are more closely linked to the input prompt, usually at the expense of poorer quality.

LogitsProcessor for classifier free guidance (CFG). The scores are split over the batch dimension, where the first half correspond to the conditional logits (predicted from the input prompt) and the second half correspond to the unconditional logits (predicted from an empty or 'null' prompt). The processor computes a weighted average across the conditional and unconditional logits, parameterised by the `guidance_scale`.

See [the paper](#) for more information.

This logits processor is exclusively compatible with [MusicGen](#)

Examples:

```
>>> from transformers import AutoProcessor, MusicgenForConditionalGeneration

>>> processor = AutoProcessor.from_pretrained("facebook/musicgen-small")
>>> model = MusicgenForConditionalGeneration.from_pretrained("facebook/musicgen-small")

>>> inputs = processor(
...     text=["80s pop track with bassy drums and synth", "90s rock song with loud guitars and heavy drums"],
...     padding=True,
...     return_tensors="pt",
... )
>>> audio_values = model.generate(**inputs, do_sample=True, guidance_scale=3, max_new_tokens=256)
```

`__call__`



(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

`class transformers.EncoderNoRepeatNGramLogitsProcessor`



(encoder_ngram_size: int, encoder_input_ids: LongTensor)

Parameters

- **encoder_ngram_size** (int) — All ngrams of size `ngram_size` can only occur within the encoder input ids.
- **encoder_input_ids** (int) — The `encoder_input_ids` that should not be repeated within the decoder ids.

LogitsProcessor that works similarly to **NoRepeatNGramLogitsProcessor**, but applied exclusively to prevent the repetition of n-grams present in the prompt.

It was designed to promote chattiness in a language model, by preventing the generation of n-grams present in previous conversation rounds.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("bigscience/bloomz-560m")
>>> tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")

>>> inputs = tokenizer("Alice: I love cats. What do you love?\nBob:", return_tensors="pt")

>>> # With greedy decoding, we see Bob repeating Alice's opinion. If Bob was a chatbot, it would be a poor one.
>>> outputs = model.generate(**inputs)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
Alice: I love cats. What do you love?
Bob: I love cats. What do you love?

>>> # With this logits processor, we can prevent Bob from repeating Alice's opinion.
>>> outputs = model.generate(**inputs, encoder_no_repeat_ngram_size=2)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
Alice: I love cats. What do you love?
Bob: My cats are very cute.
```

`__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class `transformers.EncoderRepetitionPenaltyLogitsProcessor`

(penalty: float, encoder_input_ids: LongTensor)

Parameters

- **penalty** (float) — The parameter for repetition penalty. 1.0 means no penalty. Above 1.0 rewards prompt tokens. Between 0.0 and 1.0 penalizes prompt tokens.

- **encoder_input_ids** (`torch.LongTensor`) — The `encoder_input_ids` that should be repeated within the decoder ids.

`LogitsProcessor` that works similarly to `RepetitionPenaltyLogitsProcessor`, but with an *inverse* penalty that is applied to the tokens present in the prompt. In other words, a penalty above 1.0 increases the odds of selecting tokens that were present in the prompt.

It was designed to avoid hallucination in input-grounded tasks, like summarization. Although originally intended for encoder-decoder models, it can also be used with decoder-only models like LLMs.

Examples:

```
>>> from transformers import AutoModelForCausalLM, AutoTokenizer

>>> tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")
>>> model = AutoModelForCausalLM.from_pretrained("bigscience/bloomz-560m")

>>> inputs = tokenizer(["Alice and Bob. The third member's name was"], return_tensors="pt")
>>> gen_out = model.generate(**inputs)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
Alice and Bob. The third member's name was not mentioned.

>>> # With the `encoder_repetition_penalty` argument we can trigger this logits processor in `generate`, which c
>>> # promote the use of prompt tokens ("Bob" in this example)
>>> gen_out = model.generate(**inputs, encoder_repetition_penalty=1.2)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
Alice and Bob. The third member's name was Bob. The third member's name was Bob.
```

`__call__`

(`input_ids`: `LongTensor`, `scores`: `FloatTensor`) → `torch.FloatTensor` of shape (`batch_size`, `config.vocab_size`)

Parameters

- **input_ids** (`torch.LongTensor` of shape (`batch_size`, `sequence_length`)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (`torch.FloatTensor` of shape (`batch_size`, `config.vocab_size`)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns `torch.FloatTensor` of shape (`batch_size`, `config.vocab_size`)

The processed prediction scores.

`class transformers.EpsilonLogitsWarper`

(`epsilon`: `float`, `filter_value`: `float` = `-inf`, `min_tokens_to_keep`: `int` = 1)

Parameters

- **epsilon** (`float`) — If set to > 0, only the most tokens with probabilities `epsilon` or higher are kept for generation.
- **filter_value** (`float`, *optional*, defaults to `-inf`) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (`int`, *optional*, defaults to 1) — Minimum number of tokens that cannot be filtered.

LogitsWarper that performs epsilon-sampling, i.e. restricting to tokens with `prob >= epsilon`. Takes the largest `min_tokens_to_keep` tokens if no tokens satisfy this constraint. See [Truncation Sampling as Language Model Desmoothing](#) for more information.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> set_seed(1)
>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: 1, 2", return_tensors="pt")

>>> # With sampling, the output is unexpected -- sometimes too unexpected.
>>> outputs = model.generate(**inputs, do_sample=True)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3 | < 4 (left-hand pointer) ;
<BLANKLINE>
<BLANKLINE>

>>> # With epsilon sampling, the output gets restricted to high-probability tokens. Note that this is similar to
>>> # Top P sampling, which restricts tokens based on their cumulative probability.
>>> # Pro tip: The paper recommends using 'epsilon_cutoff' values between 3e-4 and 9e-4
>>> outputs = model.generate(**inputs, do_sample=True, epsilon_cutoff=0.1)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3, 4, 5, 6, 7, 8, 9
```

`__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

`class transformers.EtaLogitsWarper`

(epsilon: float, filter_value: float = -inf, min_tokens_to_keep: int = 1, device: str = 'cpu')

Parameters

- **epsilon** (float) — A float value in the range (0, 1). Hyperparameter used to calculate the dynamic cutoff value, `eta`. The suggested values from the paper ranges from 3e-4 to 4e-3 depending on the size of the model.
- **filter_value** (float, optional, defaults to -inf) — All values that are found to be below the dynamic cutoff value, `eta`, are set to this float value. This parameter is useful when logits need to be modified for very low probability tokens that should be

excluded from generation entirely.

- **min_tokens_to_keep** (int, optional, defaults to 1) — Specifies the minimum number of tokens that must be kept for generation, regardless of their probabilities. For example, if min_tokens_to_keep is set to 1, at least one token will always be kept for generation, even if all tokens have probabilities below the cutoff eta.
- **device** (str, optional, defaults to "cpu") — The device to allocate the tensors.

LogitsWarper that performs eta-sampling, a technique to filter out tokens with probabilities below a dynamic cutoff value, eta, which is calculated based on a combination of the hyperparameter epsilon and the entropy of the token probabilities, i.e. $\text{eta} := \min(\text{epsilon}, \sqrt{\text{epsilon} * e^{-\text{entropy}(\text{probabilities})}})$. Takes the largest min_tokens_to_keep tokens if no tokens satisfy this constraint. It addresses the issue of poor quality in long samples of text generated by neural language models leading to more coherent and fluent text. See [Truncation Sampling as Language Model Desmoothing](#) for more information. Note: do_sample must be set to True for this LogitsWarper to work.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> set_seed(1)
>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: 1, 2", return_tensors="pt")

>>> # With sampling, the output is unexpected -- sometimes too unexpected.
>>> outputs = model.generate(**inputs, do_sample=True)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3 | < 4 (left-hand pointer) ;
<BLANKLINE>
<BLANKLINE>

>>> # With eta sampling, the output gets restricted to high-probability tokens. You can see it as a dynamic form
>>> # epsilon sampling that adapts its cutoff probability based on the entropy (high entropy = lower cutoff).
>>> # Pro tip: The paper recommends using 'eta_cutoff' values between 3e-4 to 4e-3
>>> outputs = model.generate(**inputs, do_sample=True, eta_cutoff=0.1)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3, 4, 5, 6, 7, 8, 9
```

__call__

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.ExponentialDecayLengthPenalty



(exponential_decay_length_penalty: Tuple, eos_token_id: Union, input_ids_seq_length: int)

Parameters

- **exponential_decay_length_penalty** (tuple(int, float)) — This tuple shall consist of: (start_index, decay_factor) where start_index indicates where penalty starts and decay_factor represents the factor of exponential decay
- **eos_token_id** (Union[int, List[int], torch.Tensor]) — The id(s) of the *end-of-sequence* token.
- **input_ids_seq_length** (int) — The length of the input sequence.

LogitsProcessor that exponentially increases the score of the eos_token_id after start_index has been reached. This allows generating shorter sequences without having a hard cutoff, allowing the eos_token to be predicted in a meaningful position.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")

>>> text = "Just wanted to let you know, I"
>>> inputs = tokenizer(text, return_tensors="pt")

>>> # Let's consider that we want short sentences, so we limit 'max_length=30'. However, we observe that the ans
>>> # tends to end abruptly.
>>> set_seed(1)
>>> outputs = model.generate(**inputs, do_sample=True, temperature=0.9, max_length=30, pad_token_id=50256)
>>> print(tokenizer.batch_decode(outputs)[0])
Just wanted to let you know, I received a link to an ebook, the book How To Start A Social Network which was
published in 2010. Although

>>> # To promote the appearance of the EOS token at the right time, we add the 'exponential_decay_length_penalty
>>> # (start_index, decay_factor)'. Instead of cutting at max_tokens, the output comes to an end before and usua
>>> # with more meaning. What happens is that starting from 'start_index' the EOS token score will be increased
>>> # by 'decay_factor' exponentially. However, if you set a high decay factor, you may also end up with abruptl
>>> # ending sequences.
>>> set_seed(1)
>>> outputs = model.generate(
...     **inputs,
...     do_sample=True,
...     temperature=0.9,
...     max_length=30,
...     pad_token_id=50256,
...     exponential_decay_length_penalty=(15, 1.6),
... )
>>> print(tokenizer.batch_decode(outputs)[0])
Just wanted to let you know, I received a link to an ebook, the book How To Start A Social Network
which<|endoftext|>

>>> # With a small decay factor, you will have a higher chance of getting a meaningful sequence.
>>> set_seed(1)
>>> outputs = model.generate(
...     **inputs,
...     do_sample=True,
...     temperature=0.9,
...     max_length=30,
...     pad_token_id=50256,
...     exponential_decay_length_penalty=(15, 1.01),
```

```
... )
>>> print(tokenizer.batch_decode(outputs)[0])
Just wanted to let you know, I received a link to an ebook, the book How To Start A Social Network which was
published in 2010.<|endoftext|>
```

`__call__`



(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

`class transformers.ForcedBOSTokenLogitsProcessor`



(bos_token_id: int)

Parameters

- **bos_token_id** (int) — The id of the token to force as the first generated token.

[LogitsProcessor](#) that enforces the specified token as the first generated token. Used with encoder-decoder models.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

>>> model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")
>>> tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")

>>> inputs = tokenizer("Translate from English to German: I love cats.", return_tensors="pt")

>>> # By default, it continues generating according to the model's logits
>>> outputs = model.generate(**inputs, max_new_tokens=10)
>>> print(tokenizer.batch_decode(outputs)[0])
<pad> Ich liebe Kitty.</s>

>>> # We can use 'forced_bos_token_id' to force the start of generation with an encoder-decoder model
>>> # (including forcing it to end straight away with an EOS token)
>>> outputs = model.generate(**inputs, max_new_tokens=10, forced_bos_token_id=tokenizer.eos_token_id)
>>> print(tokenizer.batch_decode(outputs)[0])
<pad></s>
```

`__call__`



(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.ForcedEOSTokenLogitsProcessor

(max_length: int, eos_token_id: Union, device: str = 'cpu')

Parameters

- **max_length** (int) — The maximum length of the sequence to be generated.
- **eos_token_id** (Union[int, List[int], torch.Tensor]) — The id(s) of the *end-of-sequence* token.
- **device** (str, optional, defaults to "cpu") — The device to allocate the tensors.

[LogitsProcessor](#) that enforces the specified token as the last generated token when max_length is reached.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: 1, 2, 3", return_tensors="pt")

>>> # By default, it continues generating according to the model's logits
>>> outputs = model.generate(**inputs, max_new_tokens=10)
>>> print(tokenizer.batch_decode(outputs)[0])
A sequence: 1, 2, 3, 4, 5, 6, 7, 8

>>> # 'forced_eos_token_id' ensures the generation ends with a EOS token
>>> outputs = model.generate(**inputs, max_new_tokens=10, forced_eos_token_id=tokenizer.eos_token_id)
>>> print(tokenizer.batch_decode(outputs)[0])
A sequence: 1, 2, 3, 4, 5, 6, 7,<|endoftext|>
```

__call__

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)

- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns `torch.FloatTensor` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

`class transformers.ForceTokensLogitsProcessor`



```
( force_token_map: List, _has_warned: Optional = False )
```

This processor takes a list of pairs of integers which indicates a mapping from generation indices to token indices that will be forced before generation. The processor will set their log probs to `inf` so that they are sampled at their corresponding index. Originally created for [Whisper](#).

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns `torch.FloatTensor` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

`class transformers.HammingDiversityLogitsProcessor`



```
( diversity_penalty: float, num_beams: int, num_beam_groups: int )
```

Parameters

- **diversity_penalty** (`float`) — This value is subtracted from a beam's score if it generates a token same as any beam from other group at a particular time. A higher `diversity_penalty` will enforce greater diversity among the beams. Adjusting this value can help strike a balance between diversity and natural likelihood.
- **num_beams** (`int`) — Number of beams for beam search. 1 means no beam search.
- **num_beam_groups** (`int`) — Number of groups to divide `num_beams` into in order to ensure diversity among different groups of beams. [this paper](#) for more details.

[LogitsProcessor](#) that enforces diverse beam search.

Note that this logits processor is only effective for `PreTrainedModel.group_beam_search`. See [Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models](#) for more details.

Traditional beam search often generates very similar sequences across different beams. `HammingDiversityLogitsProcessor` addresses this by penalizing beams that generate tokens already chosen by other beams in the same time step.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
>>> import torch

>>> # Initialize the model and tokenizer
>>> tokenizer = AutoTokenizer.from_pretrained("google-t5/t5-base")
>>> model = AutoModelForSeq2SeqLM.from_pretrained("google-t5/t5-base")

>>> # A long text about the solar system
>>> text = (
...     "The Solar System is a gravitationally bound system comprising the Sun and the objects that orbit it, "
...     "either directly or indirectly. Of the objects that orbit the Sun directly, the largest are the eight "
...     "planets, with the remainder being smaller objects, such as the five dwarf planets and small Solar System "
...     "bodies. The Solar System formed 4.6 billion years ago from the gravitational collapse of a giant "
...     "interstellar molecular cloud."
... )
>>> inputs = tokenizer("summarize: " + text, return_tensors="pt")

>>> # Generate diverse summary
>>> outputs_diverse = model.generate(
...     **inputs,
...     num_beam_groups=2,
...     diversity_penalty=10.0,
...     max_length=100,
...     num_beams=4,
...     num_return_sequences=2,
... )
>>> summaries_diverse = tokenizer.batch_decode(outputs_diverse, skip_special_tokens=True)

>>> # Generate non-diverse summary
>>> outputs_non_diverse = model.generate(
...     **inputs,
...     max_length=100,
...     num_beams=4,
...     num_return_sequences=2,
... )
>>> summary_non_diverse = tokenizer.batch_decode(outputs_non_diverse, skip_special_tokens=True)

>>> # With 'diversity_penalty', the resulting beams are much more diverse
>>> print(summary_non_diverse)
['the solar system formed 4.6 billion years ago from the collapse of a giant interstellar molecular cloud. of th
'the Solar System formed 4.6 billion years ago from the collapse of a giant interstellar molecular cloud. of the

>>> print(summaries_diverse)
['the solar system formed 4.6 billion years ago from the collapse of a giant interstellar molecular cloud. of th
'the solar system formed 4.6 billion years ago from the collapse of a giant interstellar molecular cloud. of the
```

 `__call__`



(input_ids: LongTensor, scores: FloatTensor, current_tokens: LongTensor, beam_group_idx: int) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search
- **current_tokens** (`torch.LongTensor` of shape `(batch_size)`) — Indices of input sequence tokens in the vocabulary, corresponding to the tokens selected by the other beam groups in the current generation step.
- **beam_group_idx** (`int`) — The index of the beam group currently being processed.

Returns `torch.FloatTensor` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

`class transformers.InfNanRemoveLogitsProcessor`

`()`

[LogitsProcessor](#) that removes all `nan` and `inf` values to avoid the generation method to fail. Note that using the logits processor should only be used if necessary since it can slow down the generation method.

This logits processor has no `generate` example, as there shouldn't be a correct combination of flags that warrants its use.

`__call__`

`( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor` of shape `(batch_size, config.vocab_size)`

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns `torch.FloatTensor` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

`class transformers.LogitNormalization`

`()`

[LogitsWarper](#) and [LogitsProcessor](#) for normalizing the scores using log-softmax. It's important to normalize the scores during beam search, after applying the logits processors or warpers, since the search algorithm used in this library doesn't do it (it only does it before, but they may need re-normalization) but it still supposes that the scores are normalized when comparing the hypotheses.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM
>>> import torch

>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: 1, 2, 3", return_tensors="pt")

>>> # By default, the scores are not normalized -- the sum of their exponentials is NOT a normalized probability
>>> # distribution, summing to 1
>>> outputs = model.generate(**inputs, return_dict_in_generate=True, output_scores=True)
>>> print(torch.allclose(torch.sum(torch.exp(outputs.scores[-1])), torch.Tensor((1.000,)), rtol=1e-4))
False

>>> # Normalizing them may have a positive impact on beam methods, or when using the scores on your application
>>> outputs = model.generate(**inputs, renormalize_logits=True, return_dict_in_generate=True, output_scores=True)
>>> print(torch.allclose(torch.sum(torch.exp(outputs.scores[-1])), torch.Tensor((1.000,)), rtol=1e-4))
True
```

`__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

`class transformers.LogitsProcessor`

()

Abstract base class for all logit processors that can be applied during generation.

`__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.LogitsProcessorList



```
( iterable = () )
```

This class can be used to create a list of [LogitsProcessor](#) or [LogitsWarper](#) to subsequently process a scores input tensor. This class inherits from list and adds a specific *call* method to apply each [LogitsProcessor](#) or [LogitsWarper](#) to the inputs.

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor, **kwargs ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search
- **kwargs** (Dict[str, Any], *optional*) — Additional kwargs that are specific to a logits processor.

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.LogitsWarper



```
( )
```

Abstract base class for all logit warpers that can be applied during generation with multinomial sampling.

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class `transformers.MinLengthLogitsProcessor`

(min_length: int, eos_token_id: Union, device: str = 'cpu')

Parameters

- **min_length** (int) — The minimum length below which the score of eos_token_id is set to -float("Inf").
- **eos_token_id** (Union[int, List[int], torch.Tensor]) — The id(s) of the *end-of-sequence* token.
- **device** (str, optional, defaults to "cpu") — The device to allocate the tensors.

LogitsProcessor enforcing a min-length by setting EOS probability to 0. Note that, for decoder-only models like most LLMs, the length includes the prompt.

Examples:

```
>>> from transformers import AutoModelForCausalLM, AutoTokenizer

>>> tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")
>>> model = AutoModelForCausalLM.from_pretrained("bigscience/bloomz-560m")

>>> inputs = tokenizer("A number:", return_tensors="pt")
>>> gen_out = model.generate(**inputs)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
A number: one

>>> # setting 'min_length' to a value smaller than the uncontrolled output length has no impact
>>> gen_out = model.generate(**inputs, min_length=3)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
A number: one

>>> # setting a larger 'min_length' will force the model to generate beyond its natural ending point, which is n
>>> # necessarily incorrect
>>> gen_out = model.generate(**inputs, min_length=10)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
A number: one thousand, nine hundred and ninety-four
```

`__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.MinNewTokensLengthLogitsProcessor



```
( prompt_length_to_skip: int, min_new_tokens: int, eos_token_id: Union, device: str = 'cpu' )
```

Parameters

- **prompt_length_to_skip** (int) — The input tokens length. Not a valid argument when used with `generate` as it will automatically assign the input length.
- **min_new_tokens** (int) — The minimum *new* tokens length below which the score of `eos_token_id` is set to `-float("Inf")`.
- **eos_token_id** (Union[int, List[int], torch.Tensor]) — The id(s) of the *end-of-sequence* token.
- **device** (str, *optional*, defaults to "cpu") — The device to allocate the tensors.

LogitsProcessor enforcing a min-length of new tokens by setting EOS (End-Of-Sequence) token probability to 0. Contrarily to MinLengthLogitsProcessor, this processor ignores the prompt.

Examples:

```
>>> from transformers import AutoModelForCausalLM, AutoTokenizer

>>> tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")
>>> model = AutoModelForCausalLM.from_pretrained("bigscience/bloomz-560m")

>>> inputs = tokenizer(["A number:"], return_tensors="pt")
>>> gen_out = model.generate(**inputs)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
A number: one

>>> # setting 'min_new_tokens' will force the model to generate beyond its natural ending point, which is not
>>> # necessarily incorrect
>>> gen_out = model.generate(**inputs, min_new_tokens=2)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
A number: one thousand
```

__call__



```
( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.MinPLogitsWarper



```
( min_p: float, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **min_p** (float) — Minimum token probability, which will be scaled by the probability of the most likely token. It must be a value between 0 and 1. Typical values are in the 0.01-0.2 range, comparably selective as setting **top_p** in the 0.99-0.8 range (use the opposite of normal **top_p** values).
- **filter_value** (float, *optional*, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, *optional*, defaults to 1) — Minimum number of tokens that cannot be filtered.

LogitsWarper that performs min-p, i.e. keeps all tokens that are above a minimum probability, scaled by the probability of the most likely token. As a result, the filter becomes more aggressive in the presence of high-probability tokens, which is a sign of a confident output that we shouldn't deviate from.

Often used together with **TemperatureLogitsWarper**. Used as an alternative to **TopPLogitsWarper** and **TopKLogitsWarper**.

Created by @menhguin and @kalomaze (github handles). Code adapted from [this external PR](#)

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> set_seed(1)
>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: 1, 2", return_tensors="pt")

>>> # With sampling, the output is unexpected -- sometimes too unexpected.
>>> outputs = model.generate(**inputs, do_sample=True)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3 | < 4 (left-hand pointer) ;
<BLANKLINE>
<BLANKLINE>

>>> # With 'min_p' sampling, the output gets restricted to high-probability tokens.
>>> # Pro tip: In practice, LLMs use 'min_p' in the 0.01-0.2 range.
>>> outputs = model.generate(**inputs, do_sample=True, min_p=0.1)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3, 4, 5, 6, 7, 8, 9
```

 `__call__`

<>

```
( input_ids: LongTensor, scores: FloatTensor )
```

 `class transformers.NoBadWordsLogitsProcessor`

<>

```
( bad_words_ids: List, eos_token_id: Union = None )
```

Parameters

- **bad_words_ids** (List[List[int]]) — List of list of token ids that are not allowed to be generated.
- **eos_token_id** (Union[int, List[int], torch.Tensor], *optional*) — The id(s) of the *end-of-sequence* token.

LogitsProcessor that enforces that specified sequences will never be selected.

In order to get the token ids of the words that should not appear in the generated text, make sure to set `add_prefix_space=True` when initializing the tokenizer, and use `tokenizer(bad_words, add_special_tokens=False).input_ids`. The `add_prefix_space` argument is only supported for some slow tokenizers, as fast tokenizers' prefixing behaviours come from pre tokenizers. Read more [here](#).

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
>>> inputs = tokenizer(["In a word, the cake is a"], return_tensors="pt")

>>> output_ids = model.generate(inputs["input_ids"], max_new_tokens=5, pad_token_id=tokenizer.eos_token_id)
>>> print(tokenizer.batch_decode(output_ids, skip_special_tokens=True)[0])
In a word, the cake is a bit of a mess.

>>> # Now let's take the bad words out. Please note that the tokenizer is initialized differently
>>> tokenizer_with_prefix_space = AutoTokenizer.from_pretrained("openai-community/gpt2", add_prefix_space=True)

>>> def get_tokens_as_list(word_list):
...     "Converts a sequence of words into a list of tokens"
...     tokens_list = []
...     for word in word_list:
...         tokenized_word = tokenizer_with_prefix_space([word], add_special_tokens=False).input_ids[0]
...         tokens_list.append(tokenized_word)
...     return tokens_list

>>> bad_words_ids = get_tokens_as_list(word_list=["mess"])
>>> output_ids = model.generate(
...     inputs["input_ids"], max_new_tokens=5, bad_words_ids=bad_words_ids, pad_token_id=tokenizer.eos_token_id
... )
>>> print(tokenizer.batch_decode(output_ids, skip_special_tokens=True)[0])
In a word, the cake is a bit of a surprise.
```

f `__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.NoRepeatNGramLogitsProcessor

```
( ngram_size: int )
```

Parameters

- **ngram_size** (int) — All ngrams of size `ngram_size` can only occur once.

N-grams are groups of “n” consecutive words, characters, or tokens taken from a sequence of text. Given the sentence: “She runs fast”, the bi-grams (n=2) would be (“she”, “runs”) and (“runs”, “fast”). In text generation, avoiding repetitions of word sequences provides a more diverse output. This [LogitsProcessor](#) enforces no repetition of n-grams by setting the scores of banned tokens to negative infinity which eliminates those tokens from consideration when further processing the scores. Note that, for decoder-only models like most LLMs, the prompt is also considered to obtain the n-grams. [Fairseq](#).

Use n-gram penalties with care. For instance, penalizing 2-grams (bigrams) in an article about the city of New York might lead to undesirable outcomes where the city’s name appears only once in the entire text. [Reference](#)

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")
>>> inputs = tokenizer(["Today I"], return_tensors="pt")

>>> output = model.generate(**inputs)
>>> print(tokenizer.decode(output[0], skip_special_tokens=True))
Today I'm not sure if I'm going to be able to do it.

>>> # Now let's add ngram size using `no_repeat_ngram_size`. This stops the repetitions ("I'm") in the output.
>>> output = model.generate(**inputs, no_repeat_ngram_size=2)
>>> print(tokenizer.decode(output[0], skip_special_tokens=True))
Today I'm not sure if I can get a better understanding of the nature of this issue
```

 [__call__](#)

<>

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

 `class transformers.PrefixConstrainedLogitsProcessor`

<>

```
( prefix_allowed_tokens_fn: Callable, num_beams: int )
```

Parameters

- **prefix_allowed_tokens_fn** (Callable[[int, torch.Tensor], List[int]]) — This function constraints the beam search to allowed tokens only at each step. This function takes 2 arguments `inputs_ids` and the batch ID `batch_id`. It has to return a list with the allowed tokens for the next generation step conditioned on the previously generated tokens `inputs_ids` and the batch ID `batch_id`.

[LogitsProcessor](#) that enforces constrained generation and is useful for prefix-conditioned constrained generation. See [Autoregressive Entity Retrieval](#) for more information.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("bigscience/bloomz-560m")
>>> tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")

>>> inputs = tokenizer("Alice and Bob", return_tensors="pt")

>>> # By default, it continues generating according to the model's logits
>>> outputs = model.generate(**inputs, max_new_tokens=5)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
Alice and Bob are friends

>>> # We can constrain it with 'prefix_allowed_tokens_fn' to force a certain behavior based on a prefix.
>>> # For instance, we can force an entire entity to be generated when its beginning is detected.
>>> entity = tokenizer(" Bob Marley", return_tensors="pt").input_ids[0] # 3 tokens
>>> def prefix_allowed_tokens_fn(batch_id, input_ids):
...     '''
...     Attempts to generate 'Bob Marley' when 'Bob' is detected.
...     In this case, 'batch_id' is not used, but you can set rules for each batch member.
...     '''
...     if input_ids[-1] == entity[0]:
...         return [entity[1].item()]
...     elif input_ids[-2] == entity[0] and input_ids[-1] == entity[1]:
...         return [entity[2].item()]
...     return list(range(tokenizer.vocab_size)) # If no match, allow all tokens

>>> outputs = model.generate(**inputs, max_new_tokens=5, prefix_allowed_tokens_fn=prefix_allowed_tokens_fn)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
Alice and Bob Marley
```

 `__call__`



(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.RepetitionPenaltyLogitsProcessor

(penalty: float)

Parameters

- **penalty** (float) — The parameter for repetition penalty. 1.0 means no penalty. Above 1.0 penalizes previously generated tokens. Between 0.0 and 1.0 rewards previously generated tokens.

LogitsProcessor that prevents the repetition of previous tokens through a penalty. This penalty is applied at most once per token. Note that, for decoder-only models like most LLMs, the considered tokens include the prompt.

In the original [paper](#), the authors suggest the use of a penalty of around 1.2 to achieve a good balance between truthful generation and lack of repetition. To penalize and reduce repetition, use `penalty` values above 1.0, where a higher value penalizes more strongly. To reward and encourage repetition, use `penalty` values between 0.0 and 1.0, where a lower value rewards more strongly.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> # Initializing the model and tokenizer for it
>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")
>>> inputs = tokenizer(["I'm not going to"], return_tensors="pt")

>>> # This shows a normal generate without any specific parameters
>>> summary_ids = model.generate(**inputs)
>>> print(tokenizer.batch_decode(summary_ids, skip_special_tokens=True)[0])
I'm not going to be able to do that. I'm going to be able to do that

>>> # This generates a penalty for repeated tokens
>>> penalized_ids = model.generate(**inputs, repetition_penalty=1.1)
>>> print(tokenizer.batch_decode(penalized_ids, skip_special_tokens=True)[0])
I'm not going to be able to do that. I'll just have to go out and play
```

f __call__

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.SequenceBiasLogitsProcessor

```
( sequence_bias: Dict )
```

Parameters

- **sequence_bias** (Dict[Tuple[int], float]) — Dictionary that maps a sequence of tokens to its bias term. Positive biases increase the odds of the sequence being selected, while negative biases do the opposite. If a sequence has a length of 1, its bias will always be applied. Otherwise, the bias will only be applied if the sequence in question is about to be completed (in the token selection step after this processor is applied).

LogitsProcessor that applies an additive bias on sequences. The bias is applied to the last token of a sequence when the next generated token can complete it. Consequently, to take the most of biasing sequences with more than one token, consider using beam methods (to gracefully work around partially completed sequences that have a negative bias) and applying the bias to their prefixes (to ensure the bias is applied earlier).

In order to get the token ids of the sequences that you want to bias, make sure to set `add_prefix_space=True` when initializing the tokenizer, and use `tokenizer(bad_words, add_special_tokens=False).input_ids`. The `add_prefix_space` argument is only supported for some slow tokenizers, as fast tokenizers' prefixing behaviours come from pre tokenizers. Read more [here](#).

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
>>> inputs = tokenizer(["The full name of Donald is Donald"], return_tensors="pt")

>>> summary_ids = model.generate(inputs["input_ids"], max_new_tokens=4)
>>> print(tokenizer.batch_decode(summary_ids, skip_special_tokens=True)[0])
The full name of Donald is Donald J. Trump Jr

>>> # Now let's control generation through a bias. Please note that the tokenizer is initialized differently!
>>> tokenizer_with_prefix_space = AutoTokenizer.from_pretrained("openai-community/gpt2", add_prefix_space=True)

>>> def get_tokens_as_tuple(word):
...     return tuple(tokenizer_with_prefix_space([word], add_special_tokens=False).input_ids[0])

>>> # If we add a negative bias without beam search, it may become "stuck" in a prefix without good continuation
>>> sequence_bias = {get_tokens_as_tuple("Trump"): -10.0}
>>> biased_ids = model.generate(inputs["input_ids"], max_new_tokens=4, sequence_bias=sequence_bias)
>>> print(tokenizer.batch_decode(biased_ids, skip_special_tokens=True)[0])
The full name of Donald is Donald J. Donald,

>>> biased_ids = model.generate(inputs["input_ids"], max_new_tokens=4, num_beams=4, sequence_bias=sequence_bias)
>>> print(tokenizer.batch_decode(biased_ids, skip_special_tokens=True)[0])
The full name of Donald is Donald Rumsfeld,

>>> # We can also add a positive bias to nudge the model towards specific tokens or continuations
>>> sequence_bias = {get_tokens_as_tuple("Donald Duck"): 10.0}
>>> biased_ids = model.generate(inputs["input_ids"], max_new_tokens=4, num_beams=4, sequence_bias=sequence_bias)
>>> print(tokenizer.batch_decode(biased_ids, skip_special_tokens=True)[0])
The full name of Donald is Donald Duck.
```

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.SuppressTokensAtBeginLogitsProcessor

(begin_suppress_tokens, begin_index, device: str = 'cpu')

[SuppressTokensAtBeginLogitsProcessor](#) suppresses a list of tokens as soon as the generate function starts generating using begin_index tokens. This should ensure that the tokens defined by begin_suppress_tokens are not generated at the beginning. Originally created for [Whisper](#).

Examples:

```
>>> from transformers import AutoProcessor, WhisperForConditionalGeneration
>>> from datasets import load_dataset

>>> processor = AutoProcessor.from_pretrained("openai/whisper-tiny.en")
>>> model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-tiny.en")
>>> ds = load_dataset("hf-internal-testing/librispeech_asr_dummy", "clean", split="validation")
>>> inputs = processor(ds[0]["audio"]["array"], return_tensors="pt")

>>> # Whisper has 'begin_suppress_tokens' set by default (= '[220, 50256]'). 50256 is the EOS token, so this mea
>>> # it can't generate and EOS token in the first iteration, but it can in the others.
>>> outputs = model.generate(**inputs, return_dict_in_generate=True, output_scores=True)
>>> print(outputs.scores[0][0, 50256])
tensor(-inf)
>>> print(outputs.scores[-1][0, 50256]) # in other places we can see some probability mass for EOS
tensor(29.9010)

>>> # If we disable 'begin_suppress_tokens', we can generate EOS in the first iteration.
>>> outputs = model.generate(
...     **inputs, return_dict_in_generate=True, output_scores=True, begin_suppress_tokens=None
... )
>>> print(outputs.scores[0][0, 50256])
tensor(11.2027)
```

__call__

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.SuppressTokensLogitsProcessor

(suppress_tokens, device: str = 'cpu')

This processor can be used to suppress a list of tokens. The processor will set their log probs to -inf so that they are not generated. Originally created for [Whisper](#).

Examples:

```
>>> from transformers import AutoProcessor, WhisperForConditionalGeneration
>>> from datasets import load_dataset

>>> processor = AutoProcessor.from_pretrained("openai/whisper-tiny.en")
>>> model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-tiny.en")
>>> ds = load_dataset("hf-internal-testing/librispeech_asr_dummy", "clean", split="validation")
>>> inputs = processor(ds[0]["audio"]["array"], return_tensors="pt")

>>> # Whisper has a long list of suppressed tokens. For instance, in this case, the token 1 is suppressed by def
>>> outputs = model.generate(**inputs, return_dict_in_generate=True, output_scores=True)
>>> print(outputs.scores[1][0, 1]) # 1 (and not 0) is the first freely generated token
tensor(-inf)

>>> # If we disable 'suppress_tokens', we can generate it.
>>> outputs = model.generate(**inputs, return_dict_in_generate=True, output_scores=True, suppress_tokens=None)
>>> print(outputs.scores[1][0, 1])
tensor(6.0678)
```

`__call__`

(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.TemperatureLogitsWarper



(temperature: float)

Parameters

- **temperature** (float) — Strictly positive float value used to modulate the logits distribution. A value smaller than 1 decreases randomness (and vice versa), with 0 being equivalent to shifting all probability mass to the most likely token.

[LogitsWarper](#) for temperature (exponential scaling output probability distribution), which effectively means that it can control the randomness of the predicted tokens. Often used together with [TopPLogitsWarper](#) and [TopKLogitsWarper](#).

Make sure that `do_sample=True` is included in the `generate` arguments otherwise the temperature value won't have any effect.

Examples:

```
>>> import torch
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> set_seed(0) # for reproducibility

>>> tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> model.config.pad_token_id = model.config.eos_token_id
>>> inputs = tokenizer(["Hugging Face Company is"], return_tensors="pt")

>>> # With temperature=1.0, the default, we consistently get random outputs due to random sampling.
>>> generate_kwargs = {"max_new_tokens": 10, "do_sample": True, "temperature": 1.0, "num_return_sequences": 2}
>>> outputs = model.generate(**inputs, **generate_kwargs)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
['Hugging Face Company is one of these companies that is going to take a',
'Hugging Face Company is a brand created by Brian A. O'Neil']

>>> # However, with temperature close to 0, it approximates greedy decoding strategies (invariant)
>>> generate_kwargs["temperature"] = 0.0001
>>> outputs = model.generate(**inputs, **generate_kwargs)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True))
['Hugging Face Company is a company that has been around for over 20 years',
'Hugging Face Company is a company that has been around for over 20 years']
```

__call__



(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class `transformers.TopKLogitsWarper`



```
( top_k: int, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **top_k** (int) — The number of highest probability vocabulary tokens to keep for top-k-filtering.
- **filter_value** (float, optional, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, optional, defaults to 1) — Minimum number of tokens that cannot be filtered.

[LogitsWarper](#) that performs top-k, i.e. restricting to the k highest probability elements. Often used together with [TemperatureLogitsWarper](#) and [TopPLogitsWarper](#).

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> set_seed(1)
>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: A, B, C, D", return_tensors="pt")

>>> # With sampling, the output is unexpected -- sometimes too unexpected.
>>> outputs = model.generate(**inputs, do_sample=True)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: A, B, C, D, E - S - O, P - R

>>> # With 'top_k' sampling, the output gets restricted the k most likely tokens.
>>> # Pro tip: In practice, LLMs use 'top_k' in the 5-50 range.
>>> outputs = model.generate(**inputs, do_sample=True, top_k=2)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: A, B, C, D, E, F, G, H, I
```

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.TopPLogitsWarper

```
( top_p: float, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **top_p** (float) — If set to < 1, only the smallest set of most probable tokens with probabilities that add up to top_p or higher are kept for generation.
- **filter_value** (float, optional, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, optional, defaults to 1) — Minimum number of tokens that cannot be filtered.

LogitsWarper that performs top-p, i.e. restricting to top tokens summing to prob_cut_off ≤ prob_cut_off. Often used together with **TemperatureLogitsWarper** and **TopKLogitsWarper**.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> set_seed(1)
>>> model = AutoModelForCausalLM.from_pretrained("distilbert/distilgpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("distilbert/distilgpt2")

>>> inputs = tokenizer("A sequence: 1, 2", return_tensors="pt")

>>> # With sampling, the output is unexpected -- sometimes too unexpected.
>>> outputs = model.generate(**inputs, do_sample=True)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3 | < 4 (left-hand pointer) ;
<BLANKLINE>
<BLANKLINE>

>>> # With 'top_p' sampling, the output gets restricted to high-probability tokens.
>>> # Pro tip: In practice, LLMs use 'top_p' in the 0.9-0.95 range.
>>> outputs = model.generate(**inputs, do_sample=True, top_p=0.1)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
A sequence: 1, 2, 3, 4, 5, 6, 7, 8, 9
```

f __call__

```
( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

class transformers.TypicalLogitsWarper

```
( mass: float = 0.9, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **mass** (float, optional, defaults to 0.9) — Value of typical_p between 0 and 1 inclusive, defaults to 0.9.
- **filter_value** (float, optional, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, optional, defaults to 1) — Minimum number of tokens that cannot be filtered.

LogitsWarper that performs typical decoding. Inspired on how humans use language, it prioritizes tokens whose log probability is close to the entropy of the token probability distribution. This means that the most likely tokens may be discarded in the process.

See [Typical Decoding for Natural Language Generation](#) for more information.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, set_seed

>>> model = AutoModelForCausalLM.from_pretrained("bigscience/bloomz-560m")
>>> tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")

>>> inputs = tokenizer("1, 2, 3", return_tensors="pt")

>>> # We can see that greedy decoding produces a sequence of numbers
>>> outputs = model.generate(**inputs)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
1, 2, 3, 4, 5, 6, 7, 8, 9, 10,

>>> # For this particular seed, we can see that sampling produces nearly the same low-information (= low entropy
>>> # sequence
>>> set_seed(18)
>>> outputs = model.generate(**inputs, do_sample=True)
>>> print(tokenizer.batch_decode(outputs, skip_special_tokens=True)[0])
1, 2, 3, 4, 5, 6, 7, 8, 9 and 10

>>> # With 'typical_p' set, the most obvious sequence is no longer produced, which may be good for your problem
>>> set_seed(18)
>>> outputs = model.generate(
...     **inputs, do_sample=True, typical_p=0.1, return_dict_in_generate=True, output_scores=True
... )
>>> print(tokenizer.batch_decode(outputs.sequences, skip_special_tokens=True)[0])
1, 2, 3 and 5

>>> # We can see that the token corresponding to "4" (token 934) in the second position, the most likely token
>>> # as seen with greedy decoding, was entirely blocked out
>>> print(outputs.scores[1][0, 934])
tensor(-inf)
```

__call__

```
( input_ids: LongTensor, scores: FloatTensor ) → torch.FloatTensor of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns `torch.FloatTensor` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

`class transformers.UnbatchedClassifierFreeGuidanceLogitsProcessor`

```
( guidance_scale: float, model, unconditional_ids: Optional = None, unconditional_attention_mask: Optional = None,
use_cache: Optional = True )
```

Parameters

- **guidance_scale** (`float`) — The guidance scale for classifier free guidance (CFG). CFG is enabled by setting `guidance_scale != 1`. Higher guidance scale encourages the model to generate samples that are more closely linked to the input prompt, usually at the expense of poorer quality. A value smaller than 1 has the opposite effect, while making the negative prompt provided with `negative_prompt_ids` (if any) act as a positive prompt.
- **model** (`PreTrainedModel`) — The model computing the unconditional scores. Supposedly the same as the one computing the conditional scores. Both models must use the same tokenizer.
- **unconditional_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) — Indices of input sequence tokens in the vocabulary for the unconditional branch. If unset, will default to the last token of the prompt.
- **unconditional_attention_mask** (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*) — Attention mask for `unconditional_ids`.
- **use_cache** (`bool`, *optional*, defaults to `True`) — Whether to cache key/values during the negative prompt forward pass.

Logits processor for Classifier-Free Guidance (CFG). The processors computes a weighted average across scores from prompt conditional and prompt unconditional (or negative) logits, parameterized by the `guidance_scale`. The unconditional scores are computed internally by prompting `model` with the `unconditional_ids` branch.

See [the paper](#) for more information.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM

>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
>>> inputs = tokenizer(["Today, a dragon flew over Paris, France,"], return_tensors="pt")
>>> out = model.generate(inputs["input_ids"], guidance_scale=1.5)
>>> tokenizer.batch_decode(out, skip_special_tokens=True)[0]
'Today, a dragon flew over Paris, France, killing at least 50 people and injuring more than 100'

>>> # with a negative prompt
>>> neg_inputs = tokenizer(["A very happy event happened,"], return_tensors="pt")
>>> out = model.generate(inputs["input_ids"], guidance_scale=2, negative_prompt_ids=neg_inputs["input_ids"])
>>> tokenizer.batch_decode(out, skip_special_tokens=True)[0]
'Today, a dragon flew over Paris, France, killing at least 130 people. French media reported that'

>>> # with a positive prompt
>>> neg_inputs = tokenizer(["A very happy event happened,"], return_tensors="pt")
```

```
>>> out = model.generate(inputs["input_ids"], guidance_scale=0, negative_prompt_ids=neg_inputs["input_ids"])
>>> tokenizer.batch_decode(out, skip_special_tokens=True)[0]
"Today, a dragon flew over Paris, France, and I'm very happy to be here. I"
```

 `__call__`

`<>`

(input_ids, scores)

 `class transformers.WhisperTimeStampLogitsProcessor`

`<>`

(generate_config, begin_index: Optional = None, _detect_timestamp_from_logprob: Optional = None)

Parameters

- **generate_config** (GenerateConfig) — The generate config used to generate the output. The following parameters are required: eos_token_id (int, optional, defaults to 50257): The id of the *end-of-sequence* token. no_timestamps_token_id (int, optional, defaults to 50363): The id of the "<|notimestamps|>" token. max_initial_timestamp_index (int, optional, defaults to 1): Used to set the maximum value of the initial timestamp. This is used to prevent the model from predicting timestamps that are too far in the future.
- **begin_index** (Optional, optional) — Token index of the first token that is generated by the model.
- **_detect_timestamp_from_logprob** (bool, optional) — Whether timestamps can be predicted from logprobs over all timestamps.

[LogitsProcessor](#) that modifies the logits for the generation of timestamps in the transcription. When the input tokens are at a specific threshold, the processor sets the scores to negative infinity. The processor makes sure that timestamp tokens appear in pairs, by masking out the logits that would break this pairing pattern. This is done to maintain the consistency and structure of generated timestamps. It also ensures that when the predicted probability of sampling any of the timestamp token is greater than any individual non-timestamp token, those non-timestamp logits are set to negative infinity. This is done to ensure the generation of timestamps over other potential tokens.

See [the paper](#) for more information.

Examples:

```
>>> import torch
>>> from transformers import AutoProcessor, WhisperForConditionalGeneration, GenerationConfig
>>> from datasets import load_dataset

>>> processor = AutoProcessor.from_pretrained("openai/whisper-tiny.en")
>>> model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-tiny.en")
>>> ds = load_dataset("hf-internal-testing/librispeech_asr_dummy", "clean", split="validation")
>>> inputs = processor(ds[3]["audio"]["array"], return_tensors="pt")
>>> input_features = inputs.input_features

>>> #Displaying timestamps
>>> generated_ids = model.generate(inputs=input_features, return_timestamps=True)
>>> transcription = processor.batch_decode(generated_ids, decode_with_timestamps=True)[0]
>>> print("Transcription:", transcription)
Transcription: <|startoftranscript|><|0.00|> He has grave doubts whether Sir Frederick Layton's work is really G

>>> #No timestamps & change EOS:
>>> #This allows the user to select a specific token to terminate the sequence on, in this case it's the word "c
>>> model.generation_config.eos_token_id = 460
>>> generated_ids = model.generate(inputs=input_features, return_timestamps=False)
```

```
>>> transcription = processor.batch_decode(generated_ids, skip_special_tokens=True)[0]
>>> print("Transcription:", transcription)
Transcription: He has grave doubts whether Sir Frederick Layton's work is really Greek after all and can
```

`__call__`



(input_ids: LongTensor, scores: FloatTensor) → torch.FloatTensor of shape (batch_size, config.vocab_size)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary. [What are input IDs?](#)
- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search

Returns torch.FloatTensor of shape (batch_size, config.vocab_size)

The processed prediction scores.

`class transformers.WatermarkLogitsProcessor`



```
( vocab_size, device, greenlist_ratio: float = 0.25, bias: float = 2.0, hashing_key: int = 15485863, seeding_scheme: str
= 'lefthash', context_width: int = 1 )
```

Expand 7 parameters

Logits processor for watermarking generated text. The processor modifies model output scores by adding a small bias to randomized set of “green” tokens before generating the next token. “Green” tokens selection process depends on the seeding_scheme used. The code was based on the [original repo](#).

The text generated by this LogitsProcessor can be detected using WatermarkDetector. See [call\(\)](#) for details,

See [the paper](#) for more information.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, WatermarkingConfig

>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> tokenizer = AutoTokenizer.from_pretrained("openai-community/gpt2")
>>> inputs = tokenizer(["Alice and Bob are"], return_tensors="pt")

>>> # normal generation
>>> out = model.generate(inputs["input_ids"], max_length=20, do_sample=False)
>>> tokenizer.batch_decode(out, skip_special_tokens=True)[0]
'Alice and Bob are both in the same room.\n\n"I\'m not sure if you\'re'

>>> # watermarked generation
>>> watermarking_config = WatermarkingConfig(bias=2.5, context_width=2, seeding_scheme="selfhash")
>>> out = model.generate(inputs["input_ids"], watermarking_config=watermarking_config, max_length=20, do_sample=
>>> tokenizer.batch_decode(out, skip_special_tokens=True)[0]
'Alice and Bob are both still alive and well and the story is pretty much a one-hour adventure'

>>> # to detect watermarked text use the WatermarkDetector class
>>> from transformers import WatermarkDetector
>>> detector = WatermarkDetector(model_config=model.config, device="cpu", watermarking_config= watermarking_conf
>>> detection_preds = detector(out)
>>> detection_preds
array([ True])
```

 `__call__`

(input_ids: LongTensor, scores: FloatTensor)

TensorFlow

 class `transformers.TFForcedBOSTokenLogitsProcessor`

(bos_token_id: int)

Parameters

- **bos_token_id** (int) — The id of the token to force as the first generated token.

[TFForcedBOSTokenLogitsProcessor](#) that enforces the specified token as the first generated token.

 `__call__`

(input_ids: Tensor, scores: Tensor, cur_len: int)

 class `transformers.TFForcedEOSTokenLogitsProcessor`

(max_length: int, eos_token_id: int)

Parameters

- **max_length** (int) — The maximum length of the sequence to be generated.
- **eos_token_id** (int) — The id of the token to force as the last generated token when max_length is reached.

TLogitsProcessor that enforces the specified token as the last generated token when `max_length` is reached.

 `__call__`



```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFForceTokensLogitsProcessor`



```
( force_token_map: List )
```

This processor takes a list of pairs of integers which indicates a mapping from generation indices to token indices that will be forced before sampling. The processor will set their log probs to 0 and all other tokens to `-inf` so that they are sampled at their corresponding index.

 `__call__`



```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFLogitsProcessor`



```
( )
```

Abstract base class for all logit processors that can be applied during generation.

 `__call__`



```
( input_ids: Tensor, scores: Tensor, cur_len: int ) → tf.Tensor of shape (batch_size, config.vocab_size)
```

Expand 4 parameters

TF method for processing logits.

 `class transformers.TFLogitsProcessorList`



```
( iterable = ( ) )
```

This class can be used to create a list of [TFLogitsProcessor](#) to subsequently process a `scores` input tensor. This class inherits from list and adds a specific *call* method to apply each [TFLogitsProcessor](#) to the inputs.

 `__call__`



```
( input_ids: Tensor, scores: Tensor, cur_len: int, **kwargs ) → tf.Tensor of shape (batch_size, config.vocab_size)
```

Expand 4 parameters

 `class transformers.TFLogitsWarper`



`( )`

Abstract base class for all logit warpers that can be applied during generation with multinomial sampling.

 `__call__`



```
( input_ids: Tensor, scores: Tensor, cur_len: int ) → tf.Tensor of shape (batch_size, config.vocab_size)
```

Expand 4 parameters

TF method for warping logits.

 `class transformers.TFMinLengthLogitsProcessor`

```
( min_length: int, eos_token_id: int )
```

Parameters

- **min_length** (int) — The minimum length below which the score of eos_token_id is set to `-float("Inf")`.
- **eos_token_id** (int) — The id of the *end-of-sequence* token.

[TFLogitsProcessor](#) enforcing a min-length by setting EOS probability to 0.

 `__call__`

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFNoBadWordsLogitsProcessor`

```
( bad_words_ids: List, eos_token_id: int )
```

Parameters

- **bad_words_ids** (List[List[int]]) — List of list of token ids that are not allowed to be generated. In order to get the tokens of the words that should not appear in the generated text, make sure to set `add_prefix_space=True` when initializing the tokenizer, and use `tokenizer(bad_words, add_special_tokens=False).input_ids`. The `add_prefix_space` argument is only supported for some slow tokenizers, as fast tokenizers' prefixing behaviours come from `pre_tokenizers`. Read more [here](#).
- **eos_token_id** (int) — The id of the *end-of-sequence* token.

[TFLogitsProcessor](#) that enforces that specified sequences will never be sampled.

 `__call__`

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFNoRepeatNGramLogitsProcessor`

```
( ngram_size: int )
```

Parameters

- **ngram_size** (int) — All ngrams of size `ngram_size` can only occur once.

[TFLogitsProcessor](#) that enforces no repetition of n-grams. See [Fairseq](#).

 `__call__`

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFRepetitionPenaltyLogitsProcessor`

```
( penalty: float )
```

Parameters

- **repetition_penalty** (float) — The parameter for repetition penalty. 1.0 means no penalty. See [this paper](#) for more details.

[TFLogitsProcessor](#) enforcing an exponential penalty on repeated sequences.

 `__call__`

<>

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFSuppressTokensAtBeginLogitsProcessor`

<>

```
( begin_suppress_tokens, begin_index )
```

[TFSuppressTokensAtBeginLogitsProcessor](#) suppresses a list of tokens as soon as the `generate` function starts generating using `begin_index` tokens. This should ensure that the tokens defined by `begin_suppress_tokens` at not sampled at the beginning of the generation.

 `__call__`

<>

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFSuppressTokensLogitsProcessor`

<>

```
( suppress_tokens )
```

This processor can be used to suppress a list of tokens. The processor will set their log probs to `-inf` so that they are not sampled.

 `__call__`

<>

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

 `class transformers.TFTemperatureLogitsWarper`

<>

```
( temperature: float )
```

Parameters

- **temperature** (float) — The value used to module the logits distribution.

[TFLogitsWarper](#) for temperature (exponential scaling output probability distribution).

 `__call__`

<>

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

class transformers.TFTopKLogitsWarper

```
( top_k: int, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **top_k** (int) — The number of highest probability vocabulary tokens to keep for top-k-filtering.
- **filter_value** (float, *optional*, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, *optional*, defaults to 1) — Minimum number of tokens that cannot be filtered.

TLogitsWarper that performs top-k, i.e. restricting to the k highest probability elements.

__call__

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

class transformers.TFTopPLogitsWarper

```
( top_p: float, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **top_p** (float) — If set to < 1, only the smallest set of most probable tokens with probabilities that add up to top_p or higher are kept for generation.
- **filter_value** (float, *optional*, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, *optional*, defaults to 1) — Minimum number of tokens that cannot be filtered.

TLogitsWarper that performs top-p, i.e. restricting to top tokens summing to <= prob_cut_off.

__call__

```
( input_ids: Tensor, scores: Tensor, cur_len: int )
```

FLAX**class transformers.FlaxForcedBOSTokenLogitsProcessor**

```
( bos_token_id: int )
```

Parameters

- **bos_token_id** (int) — The id of the token to force as the first generated token.

FlaxLogitsProcessor that enforces the specified token as the first generated token.

__call__

```
( input_ids: Array, scores: Array, cur_len: int )
```

class `transformers.FlaxForcedEOSTokenLogitsProcessor`

```
( max_length: int, eos_token_id: int )
```

Parameters

- **max_length** (int) — The maximum length of the sequence to be generated.
- **eos_token_id** (int) — The id of the token to force as the last generated token when max_length is reached.

[`FlaxLogitsProcessor`](#) that enforces the specified token as the last generated token when max_length is reached.

`__call__`

```
( input_ids: Array, scores: Array, cur_len: int )
```

class `transformers.FlaxForceTokensLogitsProcessor`

```
( force_token_map )
```

Parameters

- **force_token_map** (list) — Map giving token ids and indices where they will be forced to be sampled.

[`FlaxLogitsProcessor`](#) that takes a list of pairs of integers which indicates a mapping from generation indices to token indices that will be forced before sampling. The processor will set their log probs to 0 and all other tokens to `-inf` so that they are sampled at their corresponding index.

`__call__`

```
( input_ids: Array, scores: Array, cur_len: int )
```

class `transformers.FlaxLogitsProcessor`

```
( )
```

Abstract base class for all logit processors that can be applied during generation.

`__call__`

```
( input_ids: Array, scores: Array ) → jnp.ndarray of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (jnp.ndarray of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [`PreTrainedTokenizer`](#). See [`PreTrainedTokenizer.encode\(\)`](#) and [`PreTrainedTokenizer.call\(\)`](#) for details.

[What are input IDs?](#)

- **scores** (`jnp.ndarray` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional logits processor specific kwargs.

Returns `jnp.ndarray` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

Flax method for processing logits.

class `transformers.FlaxLogitsProcessorList`



(`iterable = ()`)

This class can be used to create a list of [FlaxLogitsProcessor](#) or [FlaxLogitsWarper](#) to subsequently process a scores input tensor. This class inherits from list and adds a specific *call* method to apply each [FlaxLogitsProcessor](#) or [FlaxLogitsWarper](#) to the inputs.

`__call__`



(`input_ids: Array`, `scores: Array`, `cur_len: int`, ***kwargs*) → `jnp.ndarray` of shape `(batch_size, config.vocab_size)`

Parameters

- **input_ids** (`jnp.ndarray` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [PreTrainedTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (`jnp.ndarray` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional logits processor specific kwargs.

Returns `jnp.ndarray` of shape `(batch_size, config.vocab_size)`

The processed prediction scores.

class `transformers.FlaxLogitsWarper`



()

Abstract base class for all logit warpers that can be applied during generation with multinomial sampling.

`__call__`



```
( input_ids: Array, scores: Array ) → jnp.ndarray of shape (batch_size, config.vocab_size)
```

Parameters

- **input_ids** (jnp.ndarray of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [PreTrainedTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (jnp.ndarray of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be logits for each vocabulary when not using beam search or log softmax for each vocabulary token when using beam search
- **kwargs** (Dict[str, Any], optional) — Additional logits processor specific kwargs.

Returns jnp.ndarray of shape (batch_size, config.vocab_size)

The processed prediction scores.

Flax method for warping logits.

class transformers.FlaxMinLengthLogitsProcessor



```
( min_length: int, eos_token_id: int )
```

Parameters

- **min_length** (int) — The minimum length below which the score of eos_token_id is set to -float("Inf").
- **eos_token_id** (int) — The id of the *end-of-sequence* token.

[FlaxLogitsProcessor](#) enforcing a min-length by setting EOS probability to 0.

`__call__`



```
( input_ids: Array, scores: Array, cur_len: int )
```

class transformers.FlaxSuppressTokensAtBeginLogitsProcessor



```
( begin_suppress_tokens, begin_index )
```

Parameters

- **begin_suppress_tokens** (List[int]) — Tokens to not sample.
- **begin_index** (int) — Index where the tokens are suppressed.

[FlaxLogitsProcessor](#) suppressing a list of tokens as soon as the generate function starts generating using begin_index tokens. This should ensure that the tokens defined by begin_suppress_tokens are not sampled at the beginning of the generation.

`__call__`



```
( input_ids, scores, cur_len: int )
```

class `transformers.FlaxSuppressTokensLogitsProcessor`

```
( suppress_tokens: list )
```

Parameters

- **suppress_tokens** (list) — Tokens to not sample.

[FlaxLogitsProcessor](#) suppressing a list of tokens at each decoding step. The processor will set their log probs to be `-inf` so they are not sampled.

`__call__`

```
( input_ids: Array, scores: Array, cur_len: int )
```

class `transformers.FlaxTemperatureLogitsWarper`

```
( temperature: float )
```

Parameters

- **temperature** (float) — The value used to module the logits distribution.

[FlaxLogitsWarper](#) for temperature (exponential scaling output probability distribution).

`__call__`

```
( input_ids: Array, scores: Array, cur_len: int )
```

class `transformers.FlaxTopKLogitsWarper`

```
( top_k: int, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **top_k** (int) — The number of highest probability vocabulary tokens to keep for top-k-filtering.
- **filter_value** (float, *optional*, defaults to `-inf`) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, *optional*, defaults to 1) — Minimum number of tokens that cannot be filtered.

[FlaxLogitsWarper](#) that performs top-k, i.e. restricting to the k highest probability elements.

`__call__`

```
( input_ids: Array, scores: Array, cur_len: int )
```

class `transformers.FlaxTopPLogitsWarper`

```
( top_p: float, filter_value: float = -inf, min_tokens_to_keep: int = 1 )
```

Parameters

- **top_p** (float) — If set to < 1, only the smallest set of most probable tokens with probabilities that add up to **top_p** or higher are kept for generation.
- **filter_value** (float, optional, defaults to -inf) — All filtered values will be set to this float value.
- **min_tokens_to_keep** (int, optional, defaults to 1) — Minimum number of tokens that cannot be filtered.

[FlaxLogitsWarper](#) that performs top-p, i.e. restricting to top tokens summing to prob_cut_off <= prob_cut_off.

 [__call__](#)

<>

```
( input_ids: Array, scores: Array, cur_len: int )
```

 class [transformers.FlaxWhisperTimeStampLogitsProcessor](#)

<>

```
( generate_config, model_config, decoder_input_length )
```

Parameters

- **generate_config** (GenerateConfig) — The generate config used to generate the output. The following parameters are required: eos_token_id (int, optional, defaults to 50257): The id of the *end-of-sequence* token. no_timestamps_token_id (int, optional, defaults to 50363): The id of the "<|notimestamps|>" token. max_initial_timestamp_index (int, optional, defaults to 1): Used to set the maximum value of the initial timestamp. This is used to prevent the model from predicting timestamps that are too far in the future.

Whisper specific Processor. This processor can be used to force a list of tokens. The processor will set their log probs to **inf** so that they are sampled at their corresponding index.

 [__call__](#)

<>

```
( input_ids, scores, cur_len )
```

StoppingCriteria

A [StoppingCriteria](#) can be used to change when to stop generation (other than EOS token). Please note that this is exclusively available to our PyTorch implementations.

 class [transformers.StoppingCriteria](#)

<>

```
( )
```

Abstract base class for all stopping criteria that can be applied during generation.

If your stopping criteria depends on the **scores** input, make sure you pass **return_dict_in_generate=True**, **output_scores=True** to **generate**.

 [__call__](#)

<>

```
( input_ids: LongTensor, scores: FloatTensor, **kwargs )
```

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [AutoTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be scores for each vocabulary token before SoftMax or scores for each vocabulary token after SoftMax. If this stopping criteria depends on the `scores` input, make sure you pass `return_dict_in_generate=True`, `output_scores=True` to generate.
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional stopping criteria specific kwargs.

class `transformers.StoppingCriteriaList`



```
( iterable = () )
```

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor, **kwargs )
```

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [AutoTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be scores for each vocabulary token before SoftMax or scores for each vocabulary token after SoftMax. If this stopping criteria depends on the `scores` input, make sure you pass `return_dict_in_generate=True`, `output_scores=True` to generate.
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional stopping criteria specific kwargs.

class `transformers.MaxLengthCriteria`



```
( max_length: int, max_position_embeddings: Optional = None )
```

Parameters

- **max_length** (`int`) — The maximum length that the output sequence can have in number of tokens.
- **max_position_embeddings** (`int`, *optional*) — The maximum model length, as defined by the model's `config.max_position_embeddings` attribute.

This class can be used to stop generation whenever the full generated number of tokens exceeds `max_length`. Keep in mind for decoder-only type of transformers, this will include the initial prompted tokens.

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor, **kwargs )
```

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [AutoTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be scores for each vocabulary token before SoftMax or scores for each vocabulary token after SoftMax. If this stopping criteria depends on the `scores` input, make sure you pass `return_dict_in_generate=True`, `output_scores=True` to generate.
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional stopping criteria specific kwargs.

class `transformers.MaxTimeCriteria`



```
( max_time: float, initial_timestamp: Optional = None )
```

Parameters

- **max_time** (`float`) — The maximum allowed time in seconds for the generation.
- **initial_time** (`float`, *optional*, defaults to `time.time()`) — The start of the generation allowed time.

This class can be used to stop generation whenever the full generation exceeds some amount of time. By default, the time will start being counted when you initialize this function. You can override this by passing an `initial_time`.

`__call__`



```
( input_ids: LongTensor, scores: FloatTensor, **kwargs )
```

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [AutoTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be scores for each vocabulary token before SoftMax or scores for each vocabulary token after SoftMax. If this stopping criteria depends on the `scores` input, make sure you pass `return_dict_in_generate=True`, `output_scores=True` to generate.
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional stopping criteria specific kwargs.

class `transformers.StopStringCriteria`



```
( tokenizer: PreTrainedTokenizerBase, stop_strings: Union )
```

Parameters

- **tokenizer** (`PreTrainedTokenizer`) — The model's associated tokenizer (necessary to extract vocab and tokenize the termination sequences)

- **stop_strings** (`Union[str, List[str]]`) — A list of strings that should end generation. If a string is passed, it will be treated like a list with a single element.

This class can be used to stop generation whenever specific string sequences are generated. It preprocesses the strings together with the tokenizer vocab to find positions where tokens can validly complete the stop strings.

Generation is stopped as soon as a token is generated that completes any of the stop strings. We want to catch any instance in which the stop string would be present in the decoded output, which means we must also catch cases with “overhangs” off one or both ends. To make this more concrete, for the stop string “stop”, any of the following token sequences would trigger the match:

- [“st”, “op”]
- [“stop”]
- [“st”, “opera”]
- [“sto”, “pper”]
- [“las”, “topper”]
- [“s”, “to”, “pped”]

Note that a match will only be triggered if the stop string is at the end of the generated sequence. In other words, these sequences will not trigger a match:

- [“stop”, “at”]
- [“st”, “op”, “at”]
- [“st”, “opera”, “tion”]

The reason these are not a match is that the stop string does not overlap with the final token. If you can remove one or more tokens from the end of the sequence without destroying the stop string, then this criterion will not match that stop string. This is by design; because this check is run after each token is generated, we can’t miss a valid stop string if one is generated, but we don’t want to halt generation just because the stop string exists somewhere in the past `input_ids`.

How is the match actually performed, though? We do it in quite a confusing way, because we want the entire match process to be compilable with Torch or XLA, which means we cannot use standard string methods. However, it is possible, with some work, to do string matching with pure tensor operations. We’ll begin by describing the algorithm we use with standard string operations, and then at the end we’ll explain how this is converted to pure tensor operations.

The key to the algorithm is an observation: Because the stop string must overlap with the end of the token sequence, we can start at the end of the sequence and work backwards. Specifically, we check that there is an overlap between the start of the final token and the end of the stop_string, or to put it another way, `stop_string[-i:] == token[:i]` for some `i > 0`. If you look at the positive examples above, you’ll see the last token in all of them fulfills this property:

- [“st”, “op”] (overlap is “op”, overlap length == 2)
- [“stop”] (overlap is “stop”, overlap length == 4)
- [“st”, “opera”] (overlap is “op”, overlap length == 2)
- [“sto”, “pper”] (overlap is “p”, overlap length == 1)
- [“las”, “topper”] (overlap is “top”, overlap length == 3)
- [“s”, “to”, “pped”] (overlap is “p”, overlap length == 1)

It's impossible to construct a matching sequence that does not have this property (feel free to verify this yourself). However, although this overlap between the start of the final token and the end of the stop string is necessary for a match, it is not sufficient. We also need to check that the rest of the token sequence is consistent with the stop string.

How do we do that? Let's use ["s", "to", "pped"] as an example. We know that the final token, "pped", has an overlap of 1 with the stop string, "stop". We then go back to the previous token, "to". Since we have already matched 1 character from the stop string, the remainder to check is "sto". We check that the next token "to" matches the end of the remainder, which it does. We have now matched 3 characters from the stop string, and the remainder to match is "s". We go back to the previous token again, which is also "s". This is a match, and so we have matched the entire stop string.

How does it work when the tokens run off the start of the stop string, though? Let's consider the example of ["las", "topper"]. The final token, "topper", has an overlap of 3 with the stop string, "stop". Therefore, the remaining stop string to match is "s". We go back to the previous token, "las". Because the remainder to match is just "s", with length 1, we consider only the final 1 character from the token, which is "s". This matches the stop string, and so the entire string is matched.

How do we compute these matches with tensor operations, though? Simply: we efficiently precompute the necessary information for all tokens! For every token, we compute:

- Its overlap with the end of the stop string, if any
- The positions inside the stop string where the token matches, including matches that run off the start.
- The total length of the token

For example, for the token "pped", we would compute an end overlap of 1, no internal matching positions, and a length of 4. For the token "to", we would compute no end overlap, a single internal matching position of 1 (counting from the end), and a length of 2. For the token "s", we would compute no end overlap, a single internal matching position of 3 (again counting from the end) and a length of 1.

As long as we have this information, we can execute the algorithm above without any string comparison operations. We simply perform the following steps:

- Check if the final token has an end-overlap with the start string
- Continue backwards, keeping track of how much of the stop string we've matched so far
- At each point, check if the next token has the current position as one of its valid positions
- Continue until either a match fails, or we completely match the whole stop string

Again, consider ["s", "to", "pped"] as an example. "pped" has an end overlap of 1, so we can begin a match. We have matched 1 character so far, so we check that the next token "to", has 1 as a valid position (again, counting from the end). It does, so we add the length of "to" to our position tracker. We have now matched 3 characters, so we check that the next token "s" has 3 as a valid position. It does, so we add its length to the position tracker. The position tracker is now 4, which is the length of the stop string. We have matched the entire stop string.

In the second case, ["las", "topper"], "topper" has an end overlap of 3, so we can begin a match. We have matched 3 characters so far, so we check that the next token "las" has 3 as a valid position. It does, because we allow tokens to match positions that run off the start of the stop string. We add its length to the position tracker. The position tracker is now 6, which is greater than the length of the stop string! Don't panic, though - this also counts as a match of the stop string. We have matched the entire stop string.

Examples:

```
>>> from transformers import AutoModelForCausalLM, AutoTokenizer

>>> tokenizer = AutoTokenizer.from_pretrained("microsoft/phi-2")
>>> model = AutoModelForCausalLM.from_pretrained("microsoft/phi-2")
>>> inputs = tokenizer("The biggest states in the USA by land area:", return_tensors="pt")

>>> gen_out = model.generate(**inputs)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
The biggest states in the USA by land area:
- Alaska
- Texas
- California

>>> # Passing one or more stop strings will halt generation after those strings are emitted
>>> # Note that generating with stop strings requires you to pass the tokenizer too
>>> gen_out = model.generate(**inputs, stop_strings=["Texas"], tokenizer=tokenizer)
>>> print(tokenizer.batch_decode(gen_out, skip_special_tokens=True)[0])
The biggest states in the USA by land area:
- Alaska
- Texas
```

`__call__`



(input_ids: LongTensor, scores: FloatTensor, **kwargs)

Parameters

- **input_ids** (torch.LongTensor of shape (batch_size, sequence_length)) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [AutoTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (torch.FloatTensor of shape (batch_size, config.vocab_size)) — Prediction scores of a language modeling head. These can be scores for each vocabulary token before SoftMax or scores for each vocabulary token after SoftMax. If this stopping criteria depends on the scores input, make sure you pass `return_dict_in_generate=True`, `output_scores=True` to generate.
- **kwargs** (Dict[str, Any], optional) — Additional stopping criteria specific kwargs.

`class transformers.EosTokenCriteria`



(eos_token_id: Union)

Parameters

- **eos_token_id** (Union[int, List[int], torch.Tensor]) — The id(s) of the *end-of-sequence* token.

This class can be used to stop generation whenever the “end-of-sequence” token is generated. By default, it uses the `model.generation_config.eos_token_id`.

`__call__`



(input_ids: LongTensor, scores: FloatTensor, **kwargs)

Parameters

- **input_ids** (`torch.LongTensor` of shape `(batch_size, sequence_length)`) — Indices of input sequence tokens in the vocabulary.

Indices can be obtained using [AutoTokenizer](#). See [PreTrainedTokenizer.encode\(\)](#) and [PreTrainedTokenizer.call\(\)](#) for details.

What are input IDs?

- **scores** (`torch.FloatTensor` of shape `(batch_size, config.vocab_size)`) — Prediction scores of a language modeling head. These can be scores for each vocabulary token before SoftMax or scores for each vocabulary token after SoftMax. If this stopping criteria depends on the `scores` input, make sure you pass `return_dict_in_generate=True`, `output_scores=True` to generate.
- **kwargs** (`Dict[str, Any]`, *optional*) — Additional stopping criteria specific kwargs.

Constraints

A [Constraint](#) can be used to force the generation to include specific tokens or sequences in the output. Please note that this is exclusively available to our PyTorch implementations.

class `transformers.Constraint`

()

Abstract base class for all constraints that can be applied during generation. It must define how the constraint can be satisfied.

All classes that inherit `Constraint` must follow the requirement that

```
completed = False
while not completed:
    _, completed = constraint.update(constraint.advance())
```

will always terminate (halt).

`advance`

() → `token_ids(torch.tensor)`

Returns `token_ids(torch.tensor)`

Must be a tensor of a list of indexable tokens, not some integer.

When called, returns the token that would take this constraint one step closer to being fulfilled.

`copy`

(`stateful = False`) → `constraint(Constraint)`

Returns `constraint(Constraint)`

The same constraint as the one being called from.

Creates a new instance of this constraint.

 **does_advance**



```
( token_id: int )
```

Reads in a token and returns whether it creates progress.

 **remaining**



```
( )
```

Returns the number of remaining steps of `advance()` in order to complete this constraint.

 **reset**



```
( )
```

Resets the state of this constraint to its initialization. We would call this in cases where the fulfillment of a constraint is abruptly by an unwanted token.

 **test**



```
( )
```

Tests whether this constraint has been properly defined.

 **update**



```
( token_id: int ) → stepped( bool )
```

Returns `stepped( bool )`

Whether this constraint has become one step closer to being fulfilled. `completed( bool )`: Whether this constraint has been completely fulfilled by this token being generated. `reset( bool )`: Whether this constraint has reset its progress by this token being generated.

Reads in a token and returns booleans that indicate the progress made by it. This function will update the state of this object unlike `does_advance(self, token_id: int)`.

This isn't to test whether a certain token will advance the progress; it's to update its state as if it has been generated. This becomes important if `token_id != desired token` (refer to `else` statement in `PhrasalConstraint`)

 **class transformers.PhrasalConstraint**



```
( token_ids: List )
```

Parameters

- **token_ids** (List[int]) — The id of the token that must be generated by the output.

Constraint enforcing that an ordered sequence of tokens is included in the output.

class transformers.DisjunctiveConstraint



```
( nested_token_ids: List )
```

Parameters

- **nested_token_ids** (List[List[int]]) — A list of words, where each word is a list of ids. This constraint is fulfilled by generating just one from the list of words.

A special **Constraint** that is fulfilled by fulfilling just one of several constraints.

class transformers.ConstraintListState



```
( constraints: List )
```

Parameters

- **constraints** (List[Constraint]) — A list of **Constraint** objects that must be fulfilled by the beam scorer.

A class for beam scorers to track its progress through a list of constraints.

advance



```
( )
```

The list of tokens to generate such that we can make progress. By “list” we don’t mean the list of token that will fully fulfill a constraint.

Given constraints $c_i = \{t_{ij} \mid j == \# \text{ of tokens}\}$, If we’re not in the middle of progressing through a specific constraint c_i , we return:

```
[t_k1 for k in indices of unfulfilled constraints]
```

If we are in the middle of a constraint, then we return: $[t_{ij}]$, where i is the index of the inprogress constraint, j is the next step for the constraint.

Though we don’t care which constraint is fulfilled first, if we are in the progress of fulfilling a constraint, that’s the only one we’ll return.

reset



```
( token_ids: Optional )
```

token_ids: the tokens generated thus far to reset the state of the progress through constraints.

BeamSearch

class transformers.BeamScorer

()

Abstract base class for all beam scorers that are used for `~PreTrainedModel.beam_search` and `~PreTrainedModel.beam_sample`.

process

(input_ids: LongTensor, next_scores: FloatTensor, next_tokens: LongTensor, next_indices: LongTensor, **kwargs) → UserDict

Expand 8 parameters

finalize

(input_ids: LongTensor, next_scores: FloatTensor, next_tokens: LongTensor, next_indices: LongTensor, max_length: int, **kwargs) → torch.LongTensor of shape (batch_size * num_return_sequences, sequence_length)

Expand 6 parameters

class transformers.BeamSearchScorer

(batch_size: int, num_beams: int, device: device, length_penalty: Optional = 1.0, do_early_stopping: Union = False, num_beam_hyps_to_keep: Optional = 1, num_beam_groups: Optional = 1, max_length: Optional = None)

Expand 8 parameters

BeamScorer implementing standard beam search decoding.

Adapted in part from [Facebook's XLM beam search code](#).

Reference for the diverse beam search algorithm and implementation [Ashwin Kalyan's DBS implementation](#)

 **process**

<>

```
( input_ids: LongTensor, next_scores: FloatTensor, next_tokens: LongTensor, next_indices: LongTensor, pad_token_id:
Union = None, eos_token_id: Union = None, beam_indices: Optional = None, group_index: Optional = 0, decoder_prompt_len:
Optional = 0 )
```

 **finalize**

<>

```
( input_ids: LongTensor, final_beam_scores: FloatTensor, final_beam_tokens: LongTensor, final_beam_indices: LongTensor,
max_length: int, pad_token_id: Union = None, eos_token_id: Union = None, beam_indices: Optional = None,
decoder_prompt_len: Optional = 0 )
```

 **class transformers.ConstrainedBeamSearchScorer**

<>

```
( batch_size: int, num_beams: int, constraints: List, device: device, length_penalty: Optional = 1.0, do_early_stopping:
Union = False, num_beam_hyps_to_keep: Optional = 1, num_beam_groups: Optional = 1, max_length: Optional = None )
```

Expand 9 parameters

BeamScorer implementing constrained beam search decoding.

process



```
( input_ids: LongTensor, next_scores: FloatTensor, next_tokens: LongTensor, next_indices: LongTensor,
scores_for_all_vocab: FloatTensor, pad_token_id: Union = None, eos_token_id: Union = None, beam_indices: Optional =
None, decoder_prompt_len: Optional = 0 ) → UserDict
```

Expand 9 parameters

finalize



```
( input_ids: LongTensor, final_beam_scores: FloatTensor, final_beam_tokens: LongTensor, final_beam_indices: LongTensor,
max_length: int, pad_token_id: Union = None, eos_token_id: Union = None, beam_indices: Optional = None,
decoder_prompt_len: Optional = 0 )
```

Streamers

class `transformers.TextStreamer`



```
( tokenizer: AutoTokenizer, skip_prompt: bool = False, **decode_kwargs )
```

Parameters

- **tokenizer** (`AutoTokenizer`) — The tokenizer used to decode the tokens.
- **skip_prompt** (`bool`, *optional*, defaults to `False`) — Whether to skip the prompt to `.generate()` or not. Useful e.g. for chatbots.
- **decode_kwargs** (`dict`, *optional*) — Additional keyword arguments to pass to the tokenizer's `decode` method.

Simple text streamer that prints the token(s) to stdout as soon as entire words are formed.

The API for the streamer classes is still under development and may change in the future.

Examples:

```
>>> from transformers import AutoModelForCausalLM, AutoTokenizer, TextStreamer

>>> tok = AutoTokenizer.from_pretrained("openai-community/gpt2")
```

```
>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> inputs = tok(["An increasing sequence: one,"], return_tensors="pt")
>>> streamer = TextStreamer(tok)

>>> # Despite returning the usual output, the streamer will also print the generated text to stdout.
>>> _ = model.generate(**inputs, streamer=streamer, max_new_tokens=20)
An increasing sequence: one, two, three, four, five, six, seven, eight, nine, ten, eleven,
```

 `end`

()

Flushes any remaining cache and prints a newline to stdout.

 `on_finalized_text`

(text: str, stream_end: bool = False)

Prints the new text to stdout. If the stream is ending, also prints a newline.

 `put`

(value)

Receives tokens, decodes them, and prints them to stdout as soon as they form entire words.

 `class transformers.TextIteratorStreamer`

(tokenizer: AutoTokenizer, skip_prompt: bool = False, timeout: Optional = None, **decode_kwargs)

Parameters

- **tokenizer** (AutoTokenizer) — The tokenizer used to decode the tokens.
- **skip_prompt** (bool, optional, defaults to False) — Whether to skip the prompt to `.generate()` or not. Useful e.g. for chatbots.
- **timeout** (float, optional) — The timeout for the text queue. If None, the queue will block indefinitely. Useful to handle exceptions in `.generate()`, when it is called in a separate thread.
- **decode_kwargs** (dict, optional) — Additional keyword arguments to pass to the tokenizer's decode method.

Streamer that stores print-ready text in a queue, to be used by a downstream application as an iterator. This is useful for applications that benefit from accessing the generated text in a non-blocking way (e.g. in an interactive Gradio demo).

The API for the streamer classes is still under development and may change in the future.

Examples:

```
>>> from transformers import AutoModelForCausalLM, AutoTokenizer, TextIteratorStreamer
>>> from threading import Thread
```

```
>>> tok = AutoTokenizer.from_pretrained("openai-community/gpt2")
>>> model = AutoModelForCausalLM.from_pretrained("openai-community/gpt2")
>>> inputs = tok(["An increasing sequence: one,"], return_tensors="pt")
>>> streamer = TextIteratorStreamer(tok)

>>> # Run the generation in a separate thread, so that we can fetch the generated text in a non-blocking way.
>>> generation_kwargs = dict(inputs, streamer=streamer, max_new_tokens=20)
>>> thread = Thread(target=model.generate, kwargs=generation_kwargs)
>>> thread.start()
>>> generated_text = ""
>>> for new_text in streamer:
...     generated_text += new_text
>>> generated_text
'An increasing sequence: one, two, three, four, five, six, seven, eight, nine, ten, eleven,'
```

on_finalized_text

```
( text: str, stream_end: bool = False )
```

Put the new text in the queue. If the stream is ending, also put a stop signal in the queue.

Caches

class transformers.Cache

```
( )
```

Base, abstract class for all caches. The actual data structure is specific to each subclass.

update

```
( key_states: Tensor, value_states: Tensor, layer_idx: int, cache_kwargs: Optional = None )
```

Parameters

- **key_states** (`torch.Tensor`) — The new key states to cache.
- **value_states** (`torch.Tensor`) — The new value states to cache.
- **layer_idx** (`int`) — The index of the layer to cache the states for.
- **cache_kwargs** (`Dict[str, Any]`, `optional`) — Additional arguments for the cache subclass. These are specific to each subclass and allow new types of cache to be created.

Updates the cache with the new `key_states` and `value_states` for the layer `layer_idx`.

class transformers.CacheConfig

```
( cache_implementation: None )
```

Base class for cache configs

update

```
( **kwargs ) → Dict[str, Any]
```

Parameters

- **kwargs** (Dict[str, Any]) — Dictionary of attributes to tentatively update this class.

Returns Dict[str, Any]

Dictionary containing all the key-value pairs that were not used to update the instance.

Updates attributes of this class instance with attributes from **kwargs** if they match existing attributes, returning all the unused kwargs.

class transformers.QuantizedCacheConfig



```
( backend: str = 'quanto', nbits: Optional = 4, axis_key: Optional = 0, axis_value: Optional = 0, q_group_size: Optional = 64, residual_length: Optional = 128, compute_dtype: Optional = torch.float16, device: Optional = 'cpu' )
```

Expand 8 parameters

Configuration class for quantized cache settings.

validate



```
( )
```

Validates if the arguments passed are correct

class transformers.DynamicCache



```
( )
```

A cache that grows dynamically as more tokens are generated. This is the default for generative models.

It stores the Key and Value states as a list of tensors, one for each layer. The expected shape for each tensor is [batch_size, num_heads, seq_len, head_dim].

 update

```
( key_states: Tensor, value_states: Tensor, layer_idx: int, cache_kwargs: Optional = None )
```

Parameters

- **key_states** (`torch.Tensor`) — The new key states to cache.
- **value_states** (`torch.Tensor`) — The new value states to cache.
- **layer_idx** (`int`) — The index of the layer to cache the states for.
- **cache_kwargs** (`Dict[str, Any]`, optional) — Additional arguments for the cache subclass. No additional arguments are used in `DynamicCache`.

Updates the cache with the new `key_states` and `value_states` for the layer `layer_idx`.

 get_seq_length

```
( layer_idx: Optional = 0 )
```

Returns the sequence length of the cached states. A layer index can be optionally passed.

 reorder_cache

```
( beam_idx: LongTensor )
```

Reorders the cache for beam search, given the selected beam indices.

 to_legacy_cache

```
( )
```

Converts the `DynamicCache` instance into the its equivalent in the legacy cache format. Used for backward compatibility.

 from_legacy_cache

```
( past_key_values: Optional = None )
```

Converts a cache in the legacy cache format into an equivalent `DynamicCache`. Used for backward compatibility.

 class transformers.QuantizedCache

```
( cache_config: QuantizedCacheConfig )
```

A quantizer cache similar to what is described in the [KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache paper](#). It allows the model to generate longer sequence length without allocating too much memory for Key and Value cache by applying quantization.

The cache has two types of storage, one for original precision and one for the quantized cache. A `residual_length` is set as a maximum capacity for the original precision cache. When the length goes beyond maximum capacity, the original precision

cache is discarded and moved into the quantized cache. The quantization is done per-channel with a set `q_group_size` for both Keys and Values, in contrast to what was described in the paper.

It stores Keys and Values a list of quantized tensors (tuples in case we need to store metadata), one for each layer. Additionally, it stores the Key and Value in original precision states as a list of tensors, one for each layer. The size of each tensor is `[batch_size, num_heads, seq_len - residual_length, head_dim]`

`update`

```
( key_states: Tensor, value_states: Tensor, layer_idx: int, cache_kwargs: Optional = None )
```

`get_seq_length`

```
( layer_idx: Optional = 0 )
```

Returns the sequence length of the cached states. A layer index can be optionally passed.

`class transformers.QuantoQuantizedCache`

```
( cache_config: CacheConfig )
```

Parameters

- **cache_config** (`QuantizedCacheConfig`,) — A configuration containing all the arguments to be used by the quantizer, including axis, qtype and group size.

Quantized Cache class that uses `quanto` as a backend to perform quantization. Current implementation supports `int2` and `int4` dtypes only.

`class transformers.HQQQuantizedCache`

```
( cache_config: CacheConfig )
```

Parameters

- **cache_config** (`QuantizedCacheConfig`,) — A configuration containing all the arguments to be used by the quantizer, including axis, qtype and group size.

Quantized Cache class that uses `HQQ` as a backend to perform quantization. Current implementation supports `int2`, `int4`, `int8` dtypes.

`class transformers.SinkCache`

```
( window_length: int, num_sink_tokens: int )
```

Parameters

- **window_length** (`int`) — The length of the context window.
- **num_sink_tokens** (`int`) — The number of sink tokens. See the original paper for more information.

A cache that as described in the [Attention Sinks paper](#). It allows the model to generate beyond the length of its context window, without losing fluency in the conversation. As it discards past tokens, the model will lose the ability to generate tokens that depend on the context that was discarded.

It stores the Key and Value states as a list of tensors, one for each layer. The expected shape for each tensor is `[batch_size, num_heads, seq_len, head_dim]`.

`update`

```
( key_states: Tensor, value_states: Tensor, layer_idx: int, cache_kwargs: Optional = None )
```

Parameters

- **key_states** (`torch.Tensor`) — The new key states to cache.
- **value_states** (`torch.Tensor`) — The new value states to cache.
- **layer_idx** (`int`) — The index of the layer to cache the states for.
- **cache_kwargs** (`Dict[str, Any]`, optional) — Additional arguments for the cache subclass. The following arguments can be used in `SinkCache`: `sin`, `cos` and `partial_rotation_size`. These arguments are used with models using RoPE, to recompute the rotation as the tokens are shifted.

Updates the cache with the new `key_states` and `value_states` for the layer `layer_idx`.

`get_seq_length`

```
( layer_idx: Optional = 0 )
```

Returns the sequence length of the cached states. A layer index can be optionally passed.

`reorder_cache`

```
( beam_idx: LongTensor )
```

Reorders the cache for beam search, given the selected beam indices.

`class transformers.StaticCache`

```
( config: PretrainedConfig, max_batch_size: int, max_cache_len: int, device, dtype = None )
```

Parameters

- **config** (`PretrainedConfig`) — The configuration file defining the shape-related attributes required to initialize the static cache.
- **max_batch_size** (`int`) — The maximum batch size with which the model will be used.
- **max_cache_len** (`int`) — The maximum sequence length with which the model will be used.
- **device** (`torch.device`) — The device on which the cache should be initialized. Should be the same as the layer.
- **dtype** (optional, defaults to `torch.float32`) — The default `dtype` to use when initializing the layer.

Static Cache class to be used with `torch.compile(model)`.

 update

```
( key_states: Tensor, value_states: Tensor, layer_idx: int, cache_kwargs: Optional = None )
```

Parameters

- **key_states** (`torch.Tensor`) — The new key states to cache.
- **value_states** (`torch.Tensor`) — The new value states to cache.
- **layer_idx** (`int`) — The index of the layer to cache the states for.
- **cache_kwargs** (`Dict[str, Any]`, optional) — Additional arguments for the cache subclass. The `StaticCache` needs the `cache_position` input to know how where to write in the cache.

Updates the cache with the new `key_states` and `value_states` for the layer `layer_idx`. It is VERY important to index using a tensor, otherwise you introduce a copy to the device.

 get_seq_length

```
( layer_idx: Optional = 0 )
```

Returns the sequence length of the cached states that were seen by the model.

 reset

```
( )
```

Resets the cache values while preserving the objects

 class transformers.EncoderDecoderCache

```
( self_attention_cache: Cache, cross_attention_cache: Cache )
```

Base, abstract class for all encoder-decoder caches. Can be used to hold combinations of self-attention and cross-attention caches.

 get_seq_length

```
( layer_idx: Optional = 0 )
```

Returns the sequence length of the cached states. A layer index can be optionally passed.

 to_legacy_cache

```
( )
```

Converts the `EncoderDecoderCache` instance into its equivalent in the legacy cache format.

 from_legacy_cache

```
( past_key_values: Optional = None )
```

Converts a cache in the legacy cache format into an equivalent `EncoderDecoderCache`.

 `reset` 

```
( )
```

 `reorder_cache` 

```
( beam_idx: LongTensor )
```

Reorders the cache for beam search, given the selected beam indices.

Watermark Utils

 `class transformers.WatermarkDetector` 

```
( model_config: PretrainedConfig, device: str, watermarking_config: Union, ignore_repeated_ngrams: bool = False,
max_cache_size: int = 128 )
```

Parameters

- **model_config** (`PretrainedConfig`) — The model config that will be used to get model specific arguments used when generating.
- **device** (`str`) — The device which was used during watermarked text generation.
- **watermarking_config** (`Union[WatermarkingConfig, Dict]`) — The exact same watermarking config and arguments used when generating text.
- **ignore_repeated_ngrams** (`bool`, *optional*, defaults to `False`) — Whether to count every unique ngram only once or not.
- **max_cache_size** (`int`, *optional*, defaults to 128) — The max size to be used for LRU caching of seeding/sampling algorithms called for every token.

Detector for detection of watermark generated text. The detector needs to be given the exact same settings that were given during text generation to replicate the watermark greenlist generation and so detect the watermark. This includes the correct device that was used during text generation, the correct watermarking arguments and the correct tokenizer vocab size. The code was based on the [original repo](#).

See [the paper](#) for more information.

Examples:

```
>>> from transformers import AutoTokenizer, AutoModelForCausalLM, WatermarkDetector, WatermarkingConfig

>>> model_id = "openai-community/gpt2"
>>> model = AutoModelForCausalLM.from_pretrained(model_id)
>>> tok = AutoTokenizer.from_pretrained(model_id)
>>> tok.pad_token_id = tok.eos_token_id
>>> tok.padding_side = "left"

>>> inputs = tok(["This is the beginning of a long story", "Alice and Bob are"], padding=True, return_tensors="p
>>> input_len = inputs["input_ids"].shape[-1]
```

```
>>> # first generate text with watermark and without
>>> watermarking_config = WatermarkingConfig(bias=2.5, seeding_scheme="selfhash")
>>> out_watermarked = model.generate(**inputs, watermarking_config=watermarking_config, do_sample=False, max_len
>>> out = model.generate(**inputs, do_sample=False, max_length=20)

>>> # now we can instantiate the detector and check the generated text
>>> detector = WatermarkDetector(model_config=model.config, device="cpu", watermarking_config=watermarking_config)
>>> detection_out_watermarked = detector(out_watermarked, return_dict=True)
>>> detection_out = detector(out, return_dict=True)
>>> detection_out_watermarked.prediction
array([ True,  True])

>>> detection_out.prediction
array([False, False])
```

`__call__`

<>

(input_ids: LongTensor, z_threshold: float = 3.0, return_dict: bool = False) → WatermarkDetectorOutput or np.array

Parameters

- **input_ids** (torch.LongTensor) — The watermark generated text. It is advised to remove the prompt, which can affect the detection.
- **z_threshold** (Dict, optional, defaults to 3.0) — Changing this threshold will change the sensitivity of the detector. Higher z threshold gives less sensitivity and vice versa for lower z threshold.
- **return_dict** (bool, optional, defaults to False) — Whether to return ~generation.WatermarkDetectorOutput or not. If not it will return boolean predictions,

Returns WatermarkDetectorOutput or np.array

A WatermarkDetectorOutput if return_dict=True otherwise a np.array.

ma

<> [Update on GitHub](#)

← [Utilities for Trainer](#)

[Utilities for Image Processors](#) →